

DIGITAL IMAGE PROCESSING

UNIT-3

Color Image Processing

Color image processing involves manipulating digital images using color information (RGB, HSV, CMY etc.) to enhance, analyze, or compress them for tasks like object recognition and computer vision. Techniques include pseudo-coloring (assigning colors to grayscale) and full-color processing (adjusting color channels), utilizing color spaces like HSI for human-like perception and RGB for display systems.

The use of color image processing is motivated by two factors:

Color is a powerful descriptor that often simplifies object identification and extraction from a scene.

Humans can discern thousands of color shades and intensities (compared to ~only 2 dozen shades of gray!).

There are two major areas of color image processing are Full-color processing and Pseudo-color processing

Full-color processing: images are acquired with a full-color sensor. Today, full-color image processing techniques are used in a broad range of applications.

Pseudo-color processing: a color is assigned to a particular monochrome intensity or range of intensities. Pseudo color (sometimes called false color) image processing consists of assigning colors to gray values based on a specified criterion. The term pseudo or false color is used to differentiate the process of assigning colors to achromatic images from the processes associated with true color images. The principal use of pseudo color is for human visualization and interpretation of grayscale events in an image or sequence of images. One of the principal motivations for using color is the fact that humans can recognize thousands of color shades and intensities, compared to less than two dozen shades of gray.

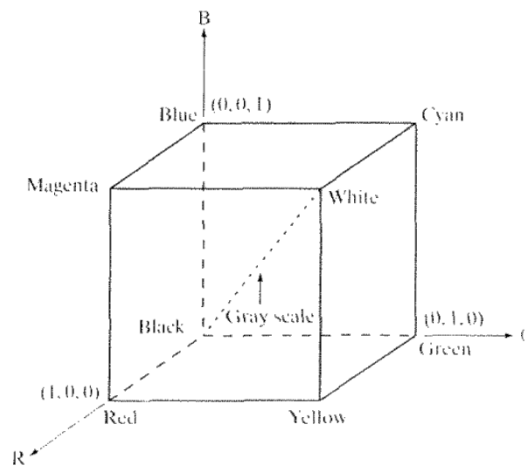
COLOR MODELS

The purpose of a color model (also called a color space or color system) is to facilitate the specification of colors in some standard way. In essence, a color

model is a specification of (1) a coordinate system, and (2) a subspace within that system, such that each color in the model is represented by a single point contained in that subspace. Most color models in use today are oriented either toward hardware (such as for color monitors and printers) or toward applications, where color manipulation is a goal (the creation of color graphics for animation is an example of the latter).

THE RGB COLOR MODEL

In the RGB model, each color appears in its primary spectral components of red, green, and blue. This model is based on a Cartesian coordinate system. The color subspace of interest is the cube shown in below Figure, in which RGB primary values are at three corners; the secondary colors cyan, magenta, and yellow are at three other corners; black is at the origin; and white is at the corner farthest from the origin.



In this model, the grayscale (points of equal RGB values) extends from black to white along the line joining these two points. The different colors in this model are points on or inside the cube, and are defined by vectors extending from the origin. For convenience, the assumption is that all color values have been normalized. All values of R, G, and B in this representation are assumed to be in the range $[0, 1]$. Note that the RGB primaries can be interpreted as unit vectors emanating from the origin of the cube.

Images represented in the RGB color model consist of three component images, one for each primary color. When fed into an RGB monitor, these three images combine on the screen to produce a composite color image. The number of bits used to represent each pixel in RGB space is called the pixel depth. Consider an RGB image in which each of the red, green, and blue images is an 8-bit image. Under these conditions, each RGB color pixel [that is, a triplet of values (R, G, B)] has a depth of 24 bits (3 image planes times the number of bits per plane). The term full-color image is used often to denote a 24-bit RGB color image. The total number of possible colors in a 24-bit RGB image is 2^{24} , or 16,777,216.



numbers representable by the number bits in the images. If, as above, the primary images are 8-bit images, the limits of the cube along each axis becomes $[0, 255]$. Then, for example, white would be at point $[255, 255, 255]$ in the cube.

Color Image Representation in Python

A color image is a group of color pixels with size of $M \times N \times 3$, where each color pixel is a triplet combination of the red, green, and blue color of a color image. The color image can be viewed as three gray-scale images that, when fed into the red, green, and blue channels of a color monitor produce a color image on the screen. The three images obtained from a color image are called the red, green, and blue component images.

The color image is defined by a data class of uint8 or uint16. If the color image is in the double class its range of values is $[0, 1]$. The number of bits required to represent the pixel values of color component images determines the bit depth of the color image.

If each color component image is an 8-bit image, the color image has a 24-bit depth. Generally, the bit depth of each component image is the same. Therefore, the bit depth of a color image is given as $(2b)^3$, where b is the number of bits. An 8-bit color image has 16,777,216 colors.

Color image's representation in Python is given here, and images are shown in the figure below

```
import cv2
import matplotlib.pyplot as plt
import tkinter as tk
from tkinter import filedialog
# 1. Open a file dialog to select the image
root = tk.Tk()
root.withdraw() # Hide the main tkinter window
file_path = filedialog.askopenfilename(title="Select the Image")
if file_path:
    # 2. Read the image
```

```

# By default, OpenCV reads in BGR format
img = cv2.imread(file_path)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
# 3. Extract Color Components
# In Python, indexing starts at 0
Red_Component = img_rgb[:, :, 0]
Green_Component = img_rgb[:, :, 1]
Blue_Component = img_rgb[:, :, 2]
# 4. Create the subplots
plt.figure(figsize=(15, 5))
# Red Component
plt.subplot(1, 3, 1)
plt.imshow(Red_Component, cmap='Reds')
plt.title('Red Component Image')
plt.axis('off')
# Green Component
plt.subplot(1, 3, 2)
plt.imshow(Green_Component, cmap='Greens')
plt.title('Green Component Image')
plt.axis('off')
# Blue Component
plt.subplot(1, 3, 3)
plt.imshow(Blue_Component, cmap='Blues')
plt.title('Blue Component Image')
plt.axis('off')
plt.tight_layout()
plt.show()
else:
    print("No file selected.")

```

Original Image

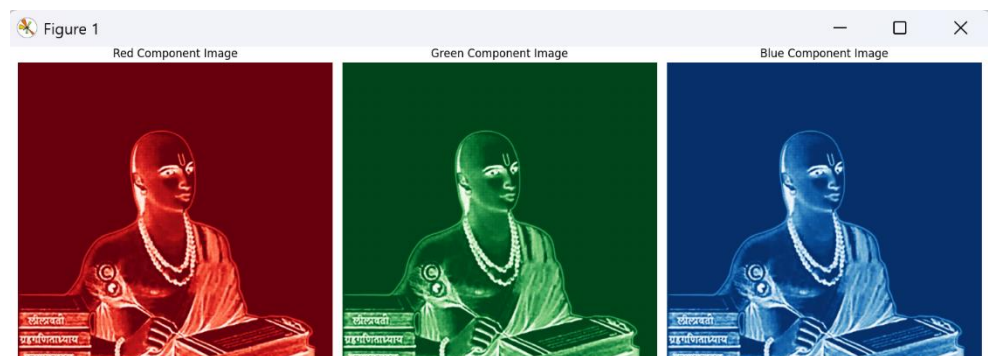


Fig: Conversion of a Gray Image into Other Spaces

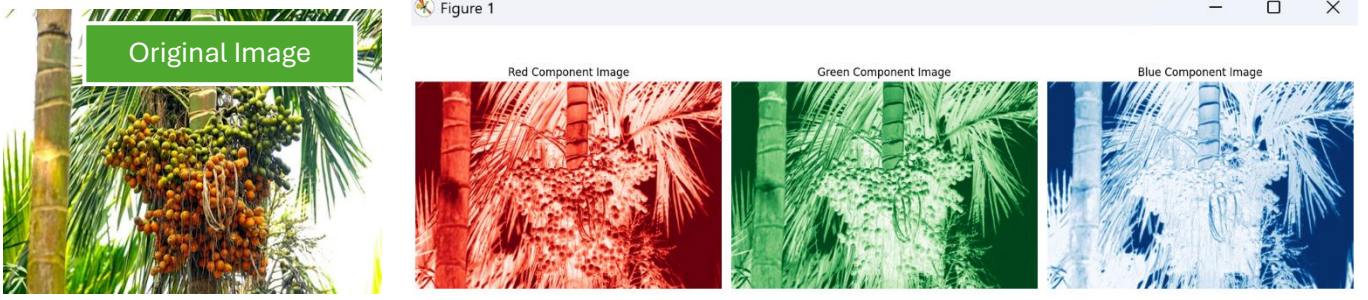


Fig: Conversion of an RGB Image into Other Spaces

Any color image can be represented by various types of color spaces. These color spaces are the NTSC (National Television System Committee) color space, YCbCr (luminance, chrominance) color space, HSV (hue, saturation, value) color space, and CMY (cyan, magenta, yellow) color space. RGB (red, green, blue) color images can be converted into any of these color spaces using Python. The inbuilt functions are available for conversion of a color image into any spaces and vice versa.

NTSC Color Space

The NTSC color space system was introduced in the USA for video transmission in the television communication system. In this color space, a color image has three components: luminance (Y), hue (I), and saturation (Q). This color space is also known as the YIQ color space. Any RGB color image can be converted into YIQ color space using Eq. 7.1

$$\begin{aligned} Y &= 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B \\ I &= 0.596 \cdot R - 0.274 \cdot G - 0.322 \cdot B \\ Q &= 0.211 \cdot R - 0.523 \cdot G + 0.312 \cdot B \end{aligned}$$

Any color image in the YIQ color space can be converted into the RGB color space using the equation given below.

$$\begin{aligned} R &= Y + 0.956 \cdot I + 0.621 \cdot Q \\ G &= Y - 0.272 \cdot I - 0.647 \cdot Q \\ B &= Y - 1.106 \cdot I + 1.703 \cdot Q \end{aligned}$$

The Python Code for the RGB color space to the NTSC color space and NTSC to RGB are as follows:

```

import cv2
import numpy as np
from tkinter import Tk, filedialog
# --- RGB to NTSC (YIQ) ---
def rgb_to_ntsc(img):
    img = img.astype(np.float32) / 255.0
    transform = np.array([
        [0.299, 0.587, 0.114],
        [0.596, -0.274, -0.322],
        [0.211, -0.523, 0.312]
    ])
    ntsc = np.dot(img, transform.T)
    return ntsc
# --- NTSC (YIQ) to RGB ---
def ntsc_to_rgb(ntsc):
    inverse_transform = np.array([
        [1.0, 0.956, 0.621],
        [1.0, -0.272, -0.647],
        [1.0, -1.106, 1.703]
    ])
    rgb = np.dot(ntsc, inverse_transform.T)
    rgb = np.clip(rgb, 0, 1)
    rgb = (rgb * 255).astype(np.uint8)
    return rgb
# --- Select image using file dialog ---
def select_image():
    root = Tk()
    root.withdraw()
    file_path = filedialog.askopenfilename(title="Select an Image")
    return file_path
# --- Main ---
def main():
    path = select_image()
    if not path:
        print("No image selected.")
        return
    # Read image (OpenCV loads as BGR)
    bgr = cv2.imread(path)
    rgb = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)
    # Convert RGB -> NTSC
    ntsc = rgb_to_ntsc(rgb)
    # Convert NTSC -> RGB
    reconstructed_rgb = ntsc_to_rgb(ntsc)
    # Convert back to BGR for OpenCV display

```

```

reconstructed_bgr = cv2.cvtColor(reconstructed_rgb, cv2.COLOR_RGB2BGR)
# Show images
cv2.imshow("Original Image", bgr)
cv2.imshow("NTSC Image", ntsc)
cv2.imshow("Reconstructed RGB from NTSC", reconstructed_bgr)
cv2.waitKey(0)
cv2.destroyAllWindows()
if __name__ == "__main__":
    main()

```

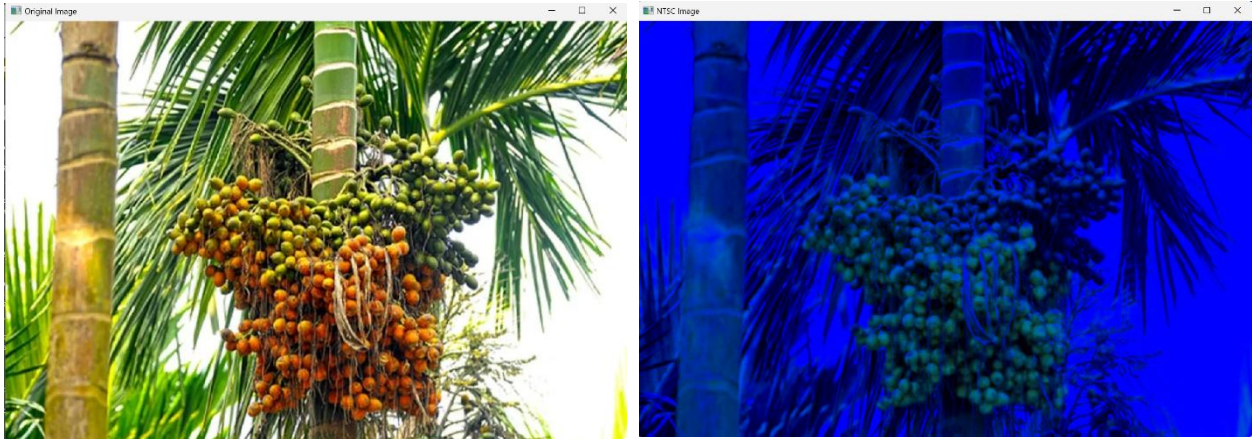


Fig: RGB image converted to NTSC model image

YCbCr Color Space

The YCbCr color space is used mainly in applications related to video. In this space, the luminance component of an image is represented by the component Y, and color information is represented by the two components Cb and Cr. Cb is the difference value between the blue channel value and a reference value, and Cr is the difference value between the red channel value and a reference value. Any RGB color image can be converted into the YCbCr color space using the equation given below.

$$Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

$$Cb = 0.564 \times (B - Y) + 0.5$$

$$Cr = 0.713 \times (R - Y) + 0.5$$

In addition, any color image in the YCbCr color space can be converted into the RGB color space using equation given below.

$$R = Y + 1.403 \times (Cr - 0.5)$$

$$G = Y - 0.344 \times (Cr - 0.5) - 0.714 \times (Cb - 0.5)$$

$$B = Y + 1.773 \times (Cb - 0.5)$$

The Python function syntax for an RGB color space converted to the YCbCr color space and YCbCr to RGB are as follows:

ycbcr = rgb_to_ycbcr(rgb)

reconstructed_rgb = ycbcr_to_rgb(ycbcr)

The Python code for a RGB color image converted into the YCbCr color space is given here and the result is shown in Figures below.

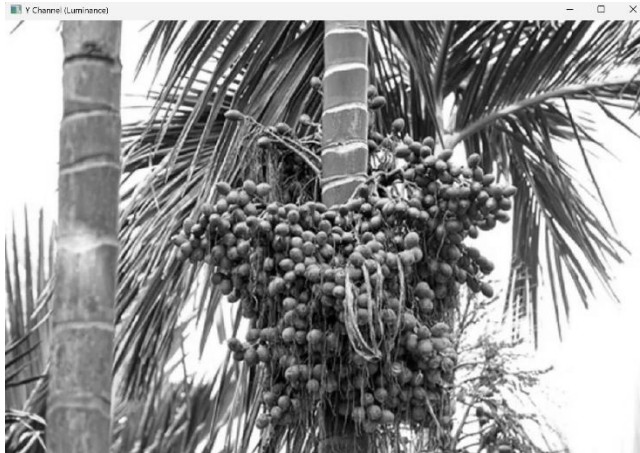


Fig: Y Channel in YCbCr Model



Fig: Cb Channel in YCbCr model



Fig: Cr Channel in YCbCr Model

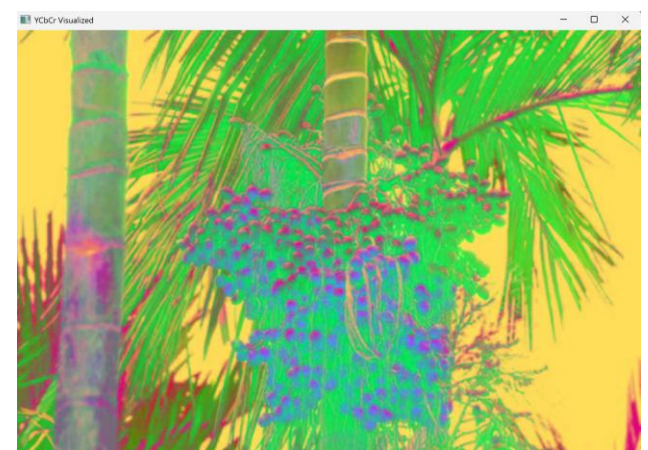


Fig: YCbCr model visualized

```
import cv2
import numpy as np
from tkinter import Tk, filedialog
# --- RGB to YCbCr ---
def rgb_to_ycbcr(img):
    img = img.astype(np.float32)
    transform = np.array([
        [0.299, 0.587, 0.114],
        [-0.168736, -0.331264, 0.5],
        [0.5, -0.418688, -0.081312]
    ])
    ycbcr = np.dot(img, transform.T)
    ycbcr[:, :, 1:] += 128.0
    return ycbcr
# --- YCbCr to RGB ---
```

```

def ycbcr_to_rgb(ycbcr):
    ycbcr = ycbcr.astype(np.float32)
    ycbcr[:, :, 1:] -= 128.0
    inverse_transform = np.array([
        [1.0, 0.0, 1.402],
        [1.0, -0.344136, -0.714136],
        [1.0, 1.772, 0.0]
    ])
    rgb = np.dot(ycbcr, inverse_transform.T)
    rgb = np.clip(rgb, 0, 255)
    return rgb.astype(np.uint8)
# --- Normalize for display ---
def normalize_channel(channel):
    return cv2.normalize(channel, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)
# --- Select image ---
def select_image():
    root = Tk()
    root.withdraw()
    return filedialog.askopenfilename(title="Select an Image")
# --- Main ---
def main():
    path = select_image()
    if not path:
        print("No image selected.")
        return
    bgr = cv2.imread(path)
    rgb = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)

    # Convert RGB -> YCbCr
    ycbcr = rgb_to_ycbcr(rgb)
    # Split channels
    Y = ycbcr[:, :, 0]
    Cb = ycbcr[:, :, 1]
    Cr = ycbcr[:, :, 2]
    # Normalize for viewing
    Y_disp = normalize_channel(Y)
    Cb_disp = normalize_channel(Cb)
    Cr_disp = normalize_channel(Cr)
    # Merge normalized channels to visualize YCbCr
    ycbcr_vis = cv2.merge([Y_disp, Cb_disp, Cr_disp])
    ycbcr_vis_bgr = cv2.cvtColor(ycbcr_vis, cv2.COLOR_RGB2BGR)
    # Convert back to RGB
    reconstructed_rgb = ycbcr_to_rgb(ycbcr)

```

```

reconstructed_bgr = cv2.cvtColor(reconstructed_rgb, cv2.COLOR_RGB2BGR)
# Show images
cv2.imshow("Original Image", bgr)
cv2.imshow("Y Channel (Luminance)", Y_disp)
cv2.imshow("Cb Channel", Cb_disp)
cv2.imshow("Cr Channel", Cr_disp)
cv2.imshow("YCbCr Visualized", ycbcr_vis_bgr)
cv2.imshow("Reconstructed RGB", reconstructed_rgb)
cv2.waitKey(0)
cv2.destroyAllWindows()
if __name__ == "__main__":
    main()

```

HSV (Hue, Saturation, Value) Color Space

Color Space HSV is a color space that is used by users to choose a specific color from a color palette. This color space is closer to the RGB color space in which a user describes a color based on experience. Any RGB color image can be converted into the HSV color space using the following equation

$$\begin{aligned}
 V &= \max(R, G, B) \\
 S &= (V - \min(R, G, B)) / V, \text{ if } V \neq 0 \\
 H &= (G - B) / 6 / S, V = R; \\
 H &= 1/2 + (B - R) / 6 / S, V = G; \\
 H &= 2/3 + (R - G) / 6 / S, V = B.
 \end{aligned}$$

The Python function syntax for an RGB color space converted into the HSV color space and HSV to RGB are as follows:

```
hsv = rgb_to_hsv(rgb)
```

```
rgb = hsv_to_rgb(hsv)
```

The below given code takes a RGB image as input and displays H, S and V channels Separately.

```

import cv2
import numpy as np
from tkinter import Tk, filedialog
# --- RGB to HSV ---
def rgb_to_hsv(img):
    img = img.astype(np.float32) / 255.0
    r, g, b = img[:, :, 0], img[:, :, 1], img[:, :, 2]
    cmax = np.max(img, axis=2)

```

```

cmin = np.min(img, axis=2)
delta = cmax - cmin
h = np.zeros_like(cmax)
# Hue
mask = delta != 0
idx = (cmax == r) & mask
h[idx] = (60 * ((g[idx] - b[idx]) / delta[idx]) + 360) % 360
idx = (cmax == g) & mask
h[idx] = (60 * ((b[idx] - r[idx]) / delta[idx]) + 120)
idx = (cmax == b) & mask
h[idx] = (60 * ((r[idx] - g[idx]) / delta[idx]) + 240)
# Saturation
s = np.zeros_like(cmax)
s[cmax != 0] = delta[cmax != 0] / cmax[cmax != 0]
# Value
v = cmax
return np.stack([h, s, v], axis=2)
# --- HSV to RGB ---
def hsv_to_rgb(hsv):
    h, s, v = hsv[:, :, 0], hsv[:, :, 1], hsv[:, :, 2]
    c = v * s
    x = c * (1 - np.abs((h / 60) % 2 - 1))
    m = v - c
    z = np.zeros_like(h)
    r1 = np.zeros_like(h)
    g1 = np.zeros_like(h)
    b1 = np.zeros_like(h)
    conds = [
        (0 <= h) & (h < 60),
        (60 <= h) & (h < 120),
        (120 <= h) & (h < 180),
        (180 <= h) & (h < 240),
        (240 <= h) & (h < 300),
        (300 <= h) & (h < 360)
    ]
    rgb_choices = [
        (c, x, z),
        (x, c, z),
        (z, c, x),
        (z, x, c),
        (x, z, c),
        (c, z, x)
    ]
    for cond, (rc, gc, bc) in zip(conds, rgb_choices):

```

```

        r1[cond] = rc[cond]
        g1[cond] = gc[cond]
        b1[cond] = bc[cond]
    r = r1 + m
    g = g1 + m
    b = b1 + m
    rgb = np.stack([r, g, b], axis=2)
    return np.clip(rgb * 255, 0, 255).astype(np.uint8)
# --- Normalize channels for display ---
def normalize(channel, scale=255):
    return np.clip(channel * scale, 0, 255).astype(np.uint8)
# --- Select image ---
def select_image():
    root = Tk()
    root.withdraw()
    return filedialog.askopenfilename(title="Select an Image")
# --- Main ---
def main():
    path = select_image()
    if not path:
        print("No image selected.")
        return
    bgr = cv2.imread(path)
    rgb = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)
    # RGB -> HSV
    hsv = rgb_to_hsv(rgb)
    H = hsv[:, :, 0] # 0-360
    S = hsv[:, :, 1] # 0-1
    V = hsv[:, :, 2] # 0-1
    # Normalize for display
    H_disp = normalize(H / 360.0) # scale to 0-255
    S_disp = normalize(S)
    V_disp = normalize(V)
    # HSV visualization (combined)
    hsv_vis = np.stack([H_disp, S_disp, V_disp], axis=2)
    hsv_vis_bgr = cv2.cvtColor(hsv_vis, cv2.COLOR_RGB2BGR)
    # HSV -> RGB
    reconstructed_rgb = hsv_to_rgb(hsv)
    reconstructed_bgr = cv2.cvtColor(reconstructed_rgb, cv2.COLOR_RGB2BGR)
    # Show outputs
    cv2.imshow("Original Image", bgr)
    cv2.imshow("Hue Channel", H_disp)
    cv2.imshow("Saturation Channel", S_disp)
    cv2.imshow("Value Channel", V_disp)

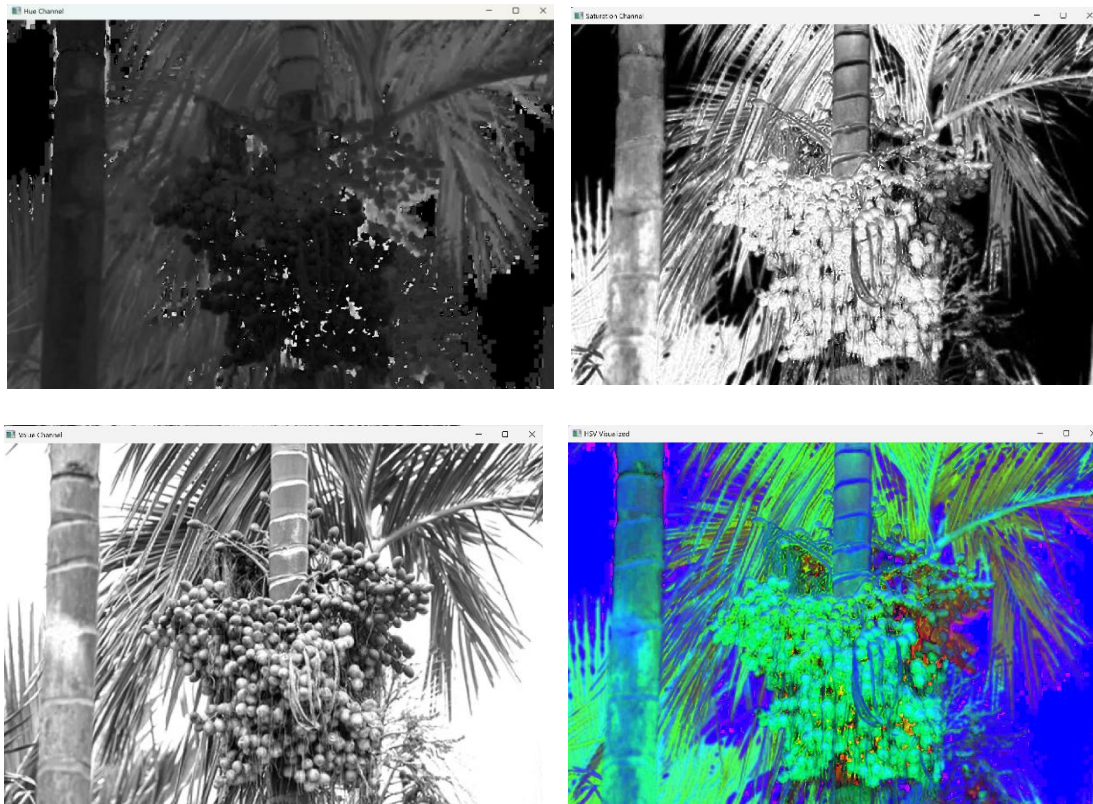
```

```

cv2.imshow("HSV Visualized", hsv_vis_bgr)
cv2.imshow("Reconstructed RGB", reconstructed_bgr)
cv2.waitKey(0)
cv2.destroyAllWindows()
if __name__ == "__main__":
    main()

```

OUTPUT:



CMY (Cyan, Magenta, Yellow) Color Space

Color Space The CMY color space represents an image using secondary colors of lights such as cyan, magenta, and yellow. The SCILAB function syntax for converting the RGB color space to the CMY color space is as follows:

cm_y_image,cyan_channel,magenta_channel,yellow_channel=
rgb_to_cmy(image_bgr)

The Python code for an RGB color image converted into the CMY color space is given here and the result is shown in Fig. 7.5.

```

import cv2
import numpy as np
def rgb_to_cmy(image_bgr):
    """

```

Converts a standard BGR image loaded by OpenCV into the CMY color space.

```

Returns both the merged CMY image and the individual C, M, and Y channels.
"""

# 1. OpenCV loads images in BGR format by default. Convert BGR to RGB.
image_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)

# 2. Normalize the RGB image to the [0, 1] range to calculate CMY precisely
# Wait, the simplest way for 8-bit images (0-255) is just subtracting from 255.
# C = 255 - R
# M = 255 - G
# Y = 255 - B
# This is equivalent to finding the mathematical inverse/negative.

# Extract RGB channels
R, G, B = cv2.split(image_rgb)
# Calculate CMY channels
C = 255 - R
M = 255 - G
Y = 255 - B
# Merge back to form the CMY image representation
# Note: OpenCV doesn't have a native "imshow CMY" renderer, so displaying
this merged
# array directly with cv2.imshow will treat it as BGR, showing weird colors.
# We output it as a merged array representing CMY matrices.
cmy_merged = cv2.merge((C, M, Y))
return cmy_merged, C, M, Y
if __name__ == "__main__":
    import os
    import sys
    # Ask the user for the image path interactively
    image_path = input("Enter the path to your image file (e.g.,
C:/Images/photo.jpg): ").strip("\\")

    if not os.path.exists(image_path):
        print(f"Error: The file '{image_path}' could not be found.")
        sys.exit(1)

    # Load the image
    image_bgr = cv2.imread(image_path)
    if image_bgr is None:
        print(f"Error: The file '{image_path}' is not a valid image format.")
        sys.exit(1)
    # Convert to CMY
    cmy_image, cyan_channel, magenta_channel, yellow_channel =
rgb_to_cmy(image_bgr)
    print("Conversion Complete!")
    print(f"Original shape: {image_bgr.shape}")
    print(f"CMY output shape: {cmy_image.shape}")
    # Save outputs directly to files so they can be securely viewed in the editor
    cv2.imwrite("output_cyan.jpg", cyan_channel)
    cv2.imwrite("output_magenta.jpg", magenta_channel)

```

```

cv2.imwrite("output_yellow.jpg", yellow_channel)
print("\nSaved channels as 'output_cyan.jpg', 'output_magenta.jpg', and
'output_yellow.jpg' in your folder.")

try:
    import matplotlib.pyplot as plt
    # Convert BGR back to RGB for proper color display in Matplotlib
    disp_img = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)

    plt.figure(figsize=(10, 8))
    plt.subplot(2, 2, 1); plt.imshow(disp_img); plt.title('Original RGB');
plt.axis('off')
    plt.subplot(2, 2, 2); plt.imshow(cyan_channel, cmap='gray'); plt.title('Cyan
(Inverse Red)'); plt.axis('off')
    plt.subplot(2, 2, 3); plt.imshow(magenta_channel, cmap='gray');
plt.title('Magenta (Inverse Green)'); plt.axis('off')
    plt.subplot(2, 2, 4); plt.imshow(yellow_channel, cmap='gray');
plt.title('Yellow (Inverse Blue)'); plt.axis('off')
    plt.tight_layout()
    plt.show() # Shows all 4 together securely
except ImportError:
    # Fallback to OpenCV standard popout windows if matplotlib is missing
    print("\nClose the OpenCV popup windows to terminate the script.")
    cv2.imshow("Original", image_bgr)
    cv2.imshow("Cyan Channel", cyan_channel)
    cv2.imshow("Magenta Channel", magenta_channel)
    cv2.imshow("Yellow Channel", yellow_channel)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

```

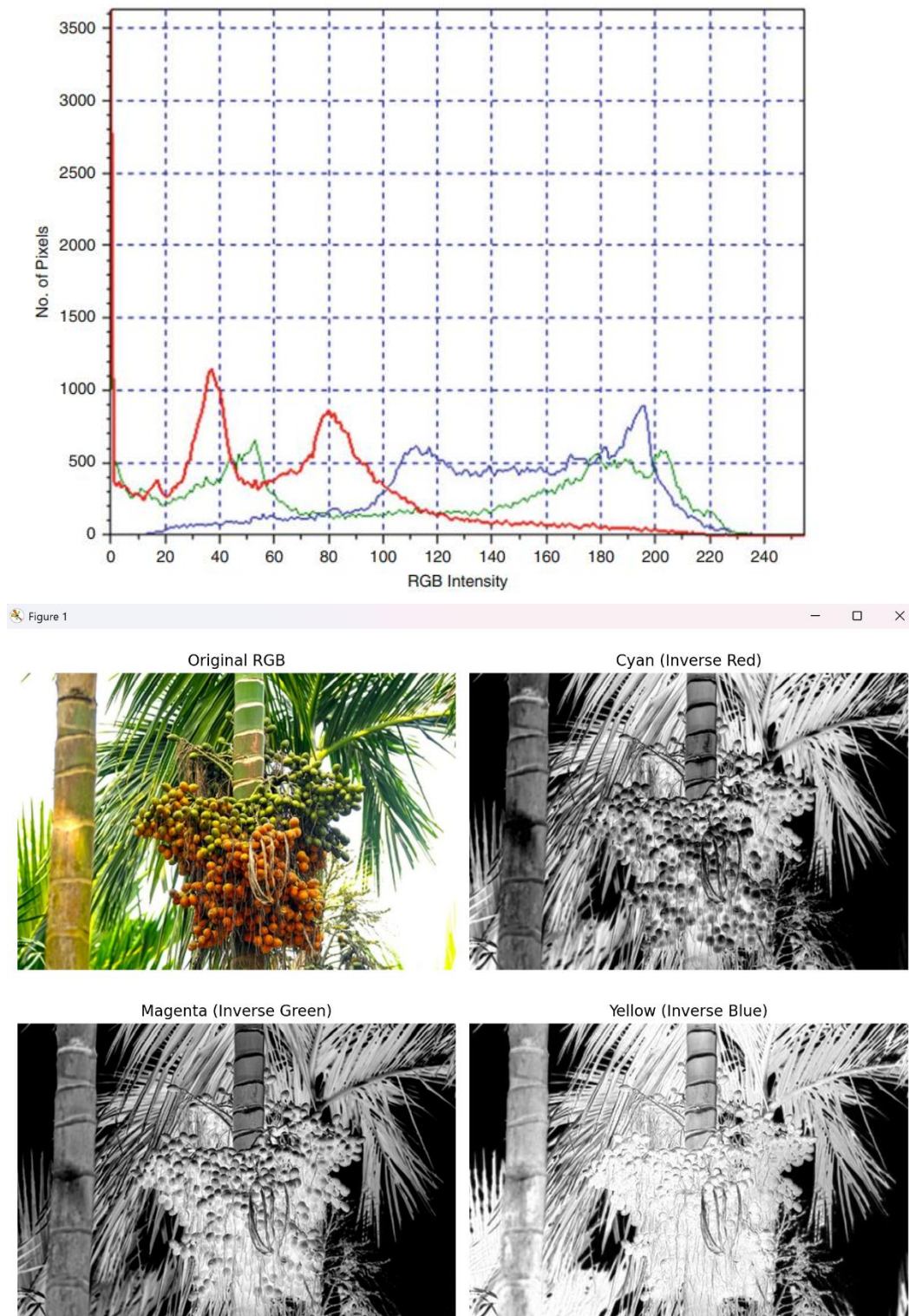


Image segmentation

Image segmentation is used to separate an image into constituent parts based on some image attributes. This process reduces a huge amount of unnecessary data, retaining only important data for image analysis. It also converts bitmap data into better structured data that are easier to interpret.

Image segmentation can be performed in two ways:

- using similarity properties of pixels and
- discontinuity properties of pixels.

The similarity properties of pixels means that targeted pixels have the same gray level intensity, while discontinuity properties of pixels means that boundary pixels have a different gray level intensity than groups of targeted pixels. The image segmentation process mainly extracts three features: lines, points, and edges in the image. The process of detection of these features is similar to mask processing.

Edge Detection

Generally, objects and the background in images have different intensities. Therefore, the edges of objects are the areas in which abrupt intensity changes occur in images. Thus, edge detection is an important process in image segmentation. The edges of an image are detected using first- and second-order derivatives. The gradient of image function $f(x, y)$ is defined as a vector, as follows

$$\nabla f = \begin{bmatrix} G_x \\ G_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

The magnitude of this vector is:

$$|\nabla f| = \left[\left(\frac{\partial f}{\partial x} \right)^2 + \left(\frac{\partial f}{\partial y} \right)^2 \right]^{1/2}$$

The fundamental property of the gradient vector is that it points in the direction of the maximum rate of change of the image vector f at coordinates (x, y) . The angle at which this maximum rate of change occurs is as follows:

$$\alpha(x, y) = \tan^{-1} \left(\frac{G_y}{G_x} \right)$$

In the edge detection algorithm, the assumption is that the edges are the pixels with a high gradient. A fast rate of change of intensity in a direction given by the angle of the gradient vector is observed in edge pixels. The various types of edge detection algorithms are available for extraction of edges in the image.

Sobel Edge Detector:

Sobel edge detection is a fundamental image processing algorithm that calculates the gradient intensity of an image to identify edges. It uses two convolutional kernels to compute approximations of the derivative—one for horizontal changes (G_x) and one for vertical (G_y). By convolving these kernels with an image, it effectively highlights areas of high spatial frequency, corresponding to edges, while providing inherent smoothing to reduce noise

The Sobel operator performs a 2-D spatial gradient measurement on an image and so emphasizes regions of high spatial frequency that correspond to edges. Typically, it is used to find the approximate absolute gradient magnitude at each point in an input grayscale image.

The operator consists of a pair of 3×3 convolution kernels as shown in Figure below. One kernel is simply the other rotated by 90°

-1	0	+1
-2	0	+2
-1	0	+1

G_x

+1	+2	+1
0	0	0
-1	-2	-1

G_y

These kernels are designed to respond maximally to edges running vertically and horizontally relative to the pixel grid, one kernel for each of the two perpendicular orientations. The kernels can be applied separately to the input image, to produce separate measurements of the gradient component in each orientation (call these G_x and G_y). These can then be combined together to find the absolute magnitude of the gradient at each point and the orientation of that gradient. The gradient magnitude is given by:

$$|G| = \sqrt{G_x^2 + G_y^2}$$

Typically, an approximate magnitude is computed using:

$$|G| = |G_x| + |G_y|$$

which is much faster to compute.

The angle of orientation of the edge (relative to the pixel grid) giving rise to the spatial gradient is given by:

$$\theta = \arctan(G_y/G_x)$$

In this case, orientation 0 is taken to mean that the direction of maximum contrast from black to white runs from left to right on the image, and other angles are measured anti-clockwise from this.

Python Code for the Sobel Edge Detector:

```
import cv2
import numpy as np
import sys
import os

def sobel_edge_detector(image_path):
    # 1. Read the image
    image = cv2.imread(image_path)
    if image is None:
        print(f'Error: Could not load the image '{image_path}'. Make sure the path is correct.')
        sys.exit(1)
    # 2. Convert the image to grayscale
    # Sobel edge detection is typically performed on a 2D grayscale intensity image
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Optional: Apply a Gaussian Blur to reduce noise before calculating gradients
    # This prevents the edge detector from picking up random high-frequency noise
    blurred_image = cv2.GaussianBlur(gray_image, (3, 3), 0)

    # 3. Apply the Sobel Operator
    # Sobel X: Detects vertical edges (horizontal gradients)
    # the depth cv2.CV_64F is used to avoid overflow when calculating derivatives
    sobel_x = cv2.Sobel(blurred_image, cv2.CV_64F, 1, 0, ksize=3)

    # Sobel Y: Detects horizontal edges (vertical gradients)
    sobel_y = cv2.Sobel(blurred_image, cv2.CV_64F, 0, 1, ksize=3)

    # 4. Convert the gradients back to an absolute 8-bit unsigned integer form
    abs_sobel_x = cv2.convertScaleAbs(sobel_x)
    abs_sobel_y = cv2.convertScaleAbs(sobel_y)

    # 5. Combine the X and Y gradients to form the final Edge map
    # We blend them together equally (0.5 weight for each)
    sobel_combined = cv2.addWeighted(abs_sobel_x, 0.5, abs_sobel_y, 0.5, 0)

    return image, gray_image, abs_sobel_x, abs_sobel_y, sobel_combined

if __name__ == "__main__":
    # Get interactive user input for the image path
    user_input = input("Enter the path to your image for Edge Detection (e.g., photo.jpg): ").strip("\\")

    if not os.path.exists(user_input):
        print("Generating a test image since no valid path was provided...")
        # Create a dummy image with distinct edges
        test_img = np.zeros((300, 300, 3), dtype=np.uint8)
        cv2.rectangle(test_img, (50, 50), (150, 150), (255, 255, 255), -1)
```

```

cv2.circle(test_img, (200, 200), 50, (255, 255, 255), -1)
cv2.imwrite("test_edge.jpg", test_img)
user_input = "test_edge.jpg"

original, grayscale, grad_x, grad_y, edges = sobel_edge_detector(user_input)

# Save the output files for easy viewing in the code editor
cv2.imwrite("sobel_x_vertical_edges.jpg", grad_x)
cv2.imwrite("sobel_y_horizontal_edges.jpg", grad_y)
cv2.imwrite("sobel_combined_edges.jpg", edges)
print("\nProcessed! Edge detection maps saved internally as:")
print("- 'sobel_x_vertical_edges.jpg'")
print("- 'sobel_y_horizontal_edges.jpg'")
print("- 'sobel_combined_edges.jpg'")

# Display the results using Matplotlib or OpenCV WaitKey
try:
    import matplotlib.pyplot as plt

    # Setup the plot
    plt.figure(figsize=(12, 8))

    plt.subplot(2, 2, 1)
    plt.imshow(cv2.cvtColor(original, cv2.COLOR_BGR2RGB))
    plt.title('Original Image')
    plt.axis('off')

    plt.subplot(2, 2, 2)
    plt.imshow(grad_x, cmap='gray')
    plt.title('Sobel X (Vertical Edges)')
    plt.axis('off')

    plt.subplot(2, 2, 3)
    plt.imshow(grad_y, cmap='gray')
    plt.title('Sobel Y (Horizontal Edges)')
    plt.axis('off')

    plt.subplot(2, 2, 4)
    plt.imshow(edges, cmap='gray')
    plt.title('Combined Sobel Edges')
    plt.axis('off')

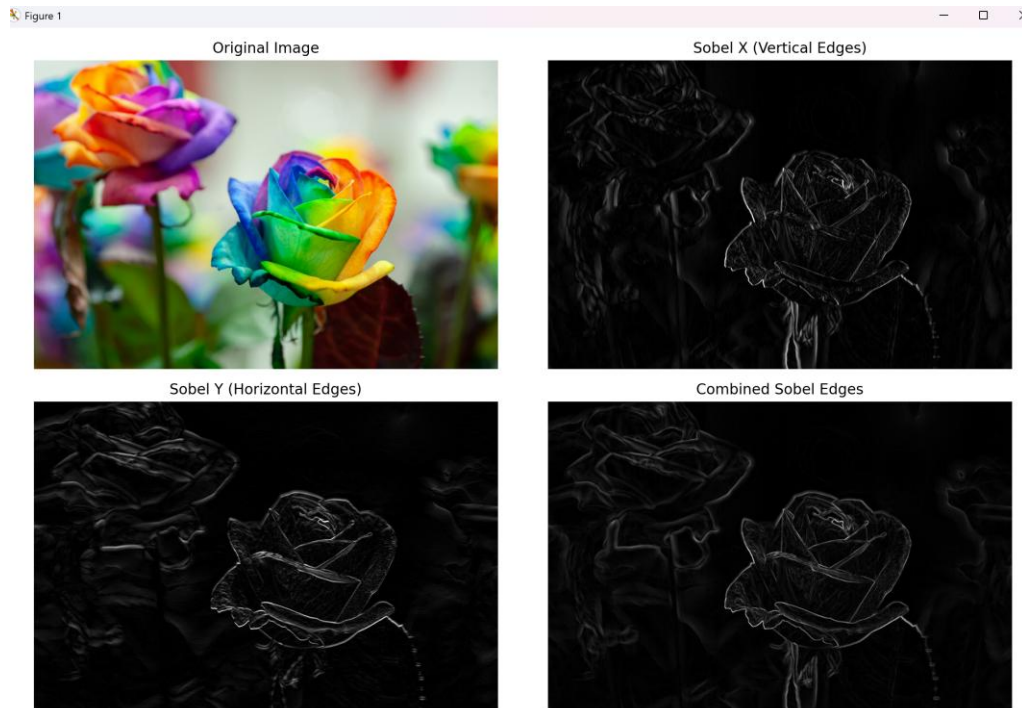
    plt.tight_layout()
    plt.show()

except ImportError:
    # Fallback to OpenCV standard popout windows if matplotlib is absent
    print("\nPlotting via Matplotlib failed. Launching OpenCV GUI... Close windows to exit.")
    cv2.imshow("Sobel X (Vertical Edges)", grad_x)

```

```
cv2.imshow("Sobel Y (Horizontal Edges)", grad_y)
cv2.imshow("Combined Sobel Edges", edges)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

OUTPUT:



Prewitt Edge Detector

Prewitt is one of the famous edge detection algorithms which use the derivative approximation to find the edges. The Prewitt operator uses a 3×3 template to calculate the values in the region, its formula is:

$$d_y = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \quad d_x = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

And the corresponding algorithms are:

Press enter or click to view image in full size

$$G(i) = |[f(i-1, j-1) + f(i-1, j) + f(i-1, j+1)] - [f(i+1, j-1) + f(i+1, j) + f(i+1, j+1)]|$$

$$G(j) = |[f(i-1, j+1) + f(i, j+1) + f(i+1, j+1)] - [f(i-1, j-1) + f(i, j-1) + f(i+1, j-1)]|$$

$$\text{And } P(i, j) = G(i) + G(j)$$

Python Code for Prewitt Edge Detection:

```
import cv2
import numpy as np
import sys
import os

def prewitt_edge_detector(image_path):
    # 1. Read the image
    image = cv2.imread(image_path)
    if image is None:
        print(f'Error: Could not load the image '{image_path}'. Make sure the path
is correct.")
        sys.exit(1)
    # 2. Convert to grayscale and apply Gaussian blur to reduce noise
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    blurred_image = cv2.GaussianBlur(gray_image, (3, 3), 0)
    # 3. Define the Prewitt kernels
    # Prewitt X: Detects vertical edges
    kernelx = np.array([[ -1, 0, 1],
                        [-1, 0, 1],
                        [-1, 0, 1]], dtype=np.float32)
    # Prewitt Y: Detects horizontal edges
    kernely = np.array([[ -1, -1, -1],
                        [ 0, 0, 0],
                        [ 1, 1, 1]], dtype=np.float32)
    # 4. Apply the Prewitt kernels to the image using a 2D filter convolution
    prewitt_x = cv2.filter2D(blurred_image, cv2.CV_64F, kernelx)
    prewitt_y = cv2.filter2D(blurred_image, cv2.CV_64F, kernely)
    # 5. Convert back to an absolute 8-bit unsigned integer scaling representation
    abs_prewitt_x = cv2.convertScaleAbs(prewitt_x)
    abs_prewitt_y = cv2.convertScaleAbs(prewitt_y)
    # 6. Combine the horizontal and vertical detected edges equally
    prewitt_combined = cv2.addWeighted(abs_prewitt_x, 0.5, abs_prewitt_y, 0.5,
0)
    return image, gray_image, abs_prewitt_x, abs_prewitt_y, prewitt_combined

if __name__ == "__main__":
    # Get interactive user input for the image path
    user_input = input("Enter the path to your image for Prewitt Edge Detection:
").strip("\\")

    if not os.path.exists(user_input):
        print("Generating a test image since no valid path was provided...")
        test_img = np.zeros((300, 300, 3), dtype=np.uint8)
        cv2.rectangle(test_img, (50, 50), (150, 150), (255, 255, 255), -1)
        cv2.circle(test_img, (200, 200), 50, (255, 255, 255), -1)
        cv2.imwrite("test_edge.jpg", test_img)
        user_input = "test_edge.jpg"

    original, grayscale, grad_x, grad_y, edges = prewitt_edge_detector(user_input)
```

```

# Save output images to workspace securely
cv2.imwrite("prewitt_x_vertical.jpg", grad_x)
cv2.imwrite("prewitt_y_horizontal.jpg", grad_y)
cv2.imwrite("prewitt_combined.jpg", edges)
print("\nProcessed! Individual detection maps saved internally as:")
print("- 'prewitt_x_vertical.jpg'")
print("- 'prewitt_y_horizontal.jpg'")
print("- 'prewitt_combined.jpg'")

# Render Output Using Matplotlib
try:
    import matplotlib.pyplot as plt

    plt.figure(figsize=(12, 8))

    plt.subplot(2, 2, 1)
    plt.imshow(cv2.cvtColor(original, cv2.COLOR_BGR2RGB))
    plt.title('Original Image')
    plt.axis('off')

    plt.subplot(2, 2, 2)
    plt.imshow(grad_x, cmap='gray')
    plt.title('Prewitt X (Vertical)')
    plt.axis('off')

    plt.subplot(2, 2, 3)
    plt.imshow(grad_y, cmap='gray')
    plt.title('Prewitt Y (Horizontal)')
    plt.axis('off')

    plt.subplot(2, 2, 4)
    plt.imshow(edges, cmap='gray')
    plt.title('Combined Prewitt Edges')
    plt.axis('off')

    plt.tight_layout()
    plt.show()

except ImportError:
    # Standard OpenCV display fallback
    print("\nPlotting via Matplotlib failed. Launching OpenCV GUI... Close
windows to exit.")
    cv2.imshow("Prewitt X", grad_x)
    cv2.imshow("Prewitt Y", grad_y)
    cv2.imshow("Combined Prewitt", edges)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

```

OUTPUT

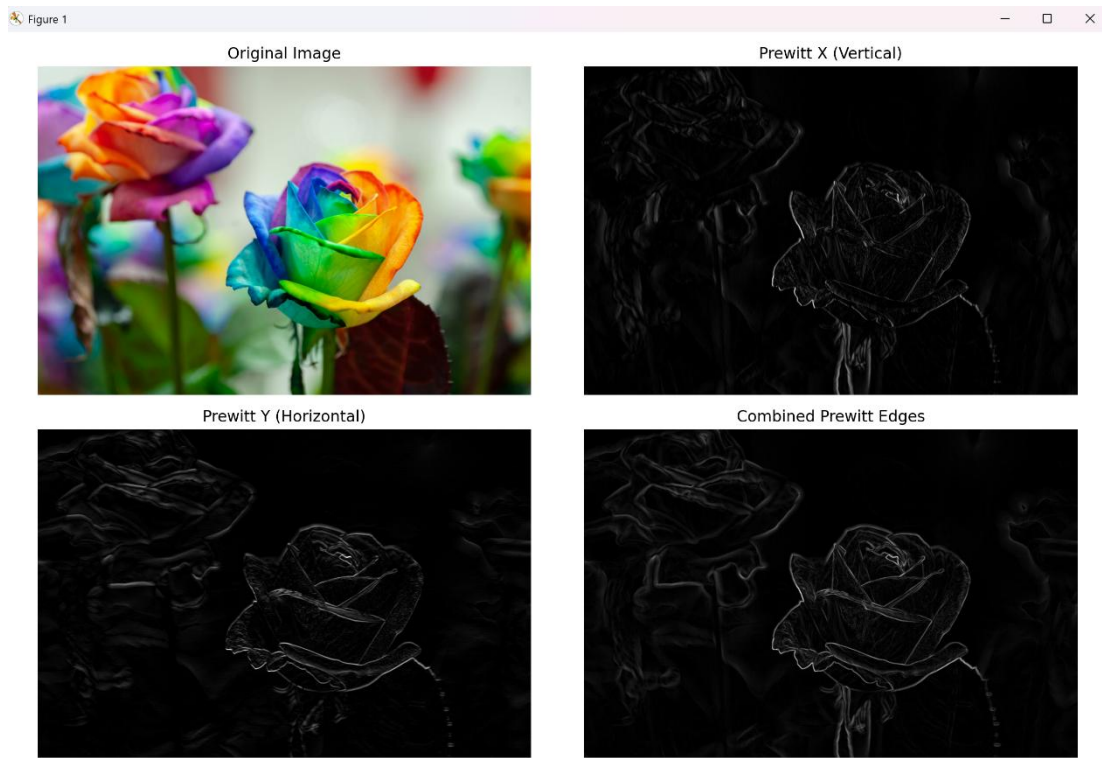


Fig: Output of Prewitt Edge Detector using Python

Roberts Edge Detector

The Roberts edge detector is the simplest edge detector algorithm, which was proposed and implemented by Lawrence Roberts in 1963. The Roberts Edge Detector (or Roberts Cross operator) is a 2D spatial gradient operator used in image processing to detect edges by calculating the sum of squares of differences between diagonally adjacent pixels. It uses small 2x2 convolution kernels, making it computationally fast, efficient, and well-suited for highlighting high-frequency, sharp, or diagonal edges.

Key Characteristics and Details:

Methodology: The operator computes the gradient magnitude using a pair of 2x2 kernels (masks) that are designed to respond to edges at 45° or 135° to the pixel grid. The masks can be applied to the input image independently to produce separate gradient component measurements in each orientation.

Mask Kernels:

$$G_x = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

$$G_y = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

These can then be combined to determine the absolute magnitude and orientation of the gradient at each site. The gradient magnitude is given by

$$G = \sqrt{G_x^2 + G_y^2}$$

The angle of orientation of the edge giving rise to the spatial gradient (relative to the pixel grid orientation) is given by:

$$\theta = \arctan\left(\frac{G_x}{G_y}\right) - \frac{3\pi}{4}$$

In this case, orientation θ is taken to mean that the direction of maximum contrast from black to white runs from left to right on the image, and other angles are measured anticlockwise from this.

Advantages: Its small 2x2 mask makes it very quick to compute compared to 3x3 methods like Sobel or Prewitt. It excels at detecting sharp, diagonal changes in intensity.

Disadvantages: It is highly sensitive to noise because the small mask only uses a limited, local neighbourhood to calculate derivatives. It also produces weak responses to genuine edges that are not very sharp.

Applications: Primarily used for fast edge detection where speed is critical and the image has low noise.

Python Code for Roberts Edge Detector

```
import cv2
import numpy as np
import sys
import os
import tkinter as tk
from tkinter import filedialog

def roberts_edge_detector(image_path):
    # 1. Read the image
    image = cv2.imread(image_path)

    if image is None:
        print(f'Error: Could not load the image '{image_path}'. Make sure the file isn't corrupted.')
        sys.exit(1)

    # 2. Convert to grayscale and apply slight Gaussian blur to reduce noise
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    blurred_image = cv2.GaussianBlur(gray_image, (3, 3), 0)

    # Needs to be a float type for convolution calculation without clipping negatives
    img_float = np.asarray(blurred_image, dtype=np.float64)

    # 3. Define the Roberts Cross kernels (2x2 matrices)
    # Roberts X (cross-1 - basically diagonal edge 1)
    kernelx = np.array([[ 1,  0],
                        [ 0, -1]], dtype=np.float64)
```

```

# Roberts Y (cross-2 - basically diagonal edge 2)
kernely = np.array([[ 0,  1],
                    [-1,  0]], dtype=np.float64)

# 4. Convolve the image with the Roberts Kernels using OpenCV Native filter2D
# This successfully bypasses the crashing Scipy dependency!
roberts_x = cv2.filter2D(img_float, cv2.CV_64F, kernelx)
roberts_y = cv2.filter2D(img_float, cv2.CV_64F, kernely)

# 5. Bring values back into an absolute positive range and scale to 8-bit
unsigned integer form
# We must normalize because Roberts outputs can theoretically range quite
wide.
abs_roberts_x = cv2.convertScaleAbs(roberts_x)
abs_roberts_y = cv2.convertScaleAbs(roberts_y)

# Calculate the exact gradient magnitude  $\sqrt{x^2 + y^2}$ 
# (cv2.addWeighted works as an approximation, but this is the formal
mathematical formula)
gradient_magnitude = np.sqrt(np.square(roberts_x) + np.square(roberts_y))
gradient_magnitude *= 255.0 / gradient_magnitude.max() # Normalize
roberts_combined = np.uint8(gradient_magnitude)

return image, gray_image, abs_roberts_x, abs_roberts_y, roberts_combined

def open_file_dialog():
    """Opens a native OS file upload dialog box securely to get an image path"""
    print("Launching File Upload Dialog Box...")
    # Initialize hidden tkinter window
    root = tk.Tk()
    root.withdraw()

    # Force the window to top level to ensure user sees it
    root.attributes('-topmost', True)

    # Launch file browser specifically filtering for known image types
    file_path = filedialog.askopenfilename(
        title="Select an Input Image",
        filetypes=[
            ("Image Files", "*.png *.jpg *.jpeg *.bmp *.tiff"),
            ("All Files", "*.*")
        ]
    )

    # Clean up the tkinter hidden engine
    root.destroy()
    return file_path

if __name__ == "__main__":

```

```

# Get user input via GUI File Upload Dialog Box!
user_input = open_file_dialog()

if not user_input or not os.path.exists(user_input):
    print("No valid file selected. Generating a generic square shape to test
detection...")
    test_img = np.zeros((300, 300, 3), dtype=np.uint8)
    cv2.rectangle(test_img, (50, 50), (150, 150), (255, 255, 255), -1)
    cv2.circle(test_img, (200, 200), 50, (255, 255, 255), -1)
    # Add diagonal line since Roberts is specifically good at diagonals
    cv2.line(test_img, (150, 50), (250, 150), (255, 255, 255), 3)
    cv2.imwrite("test_edge.jpg", test_img)
    user_input = "test_edge.jpg"

print(f"Processing image: {user_input}")
original, grayscale, grad_x, grad_y, edges = roberts_edge_detector(user_input)

# Output File Saving
cv2.imwrite("roberts_x_diagonal.jpg", grad_x)
cv2.imwrite("roberts_y_diagonal.jpg", grad_y)
cv2.imwrite("roberts_combined.jpg", edges)
print("\nProcessed! Edge detection maps saved internally as:")
print("- 'roberts_x_diagonal.jpg'")
print("- 'roberts_y_diagonal.jpg'")
print("- 'roberts_combined.jpg'")

# Graphic Plot Output display
try:
    import matplotlib.pyplot as plt

    plt.figure(figsize=(12, 8))

    plt.subplot(2, 2, 1)
    plt.imshow(cv2.cvtColor(original, cv2.COLOR_BGR2RGB))
    plt.title('Original Target Image')
    plt.axis('off')

    plt.subplot(2, 2, 2)
    plt.imshow(grad_x, cmap='gray')
    plt.title('Roberts X (Diagonal 1)')
    plt.axis('off')

    plt.subplot(2, 2, 3)
    plt.imshow(grad_y, cmap='gray')
    plt.title('Roberts Y (Diagonal 2)')
    plt.axis('off')

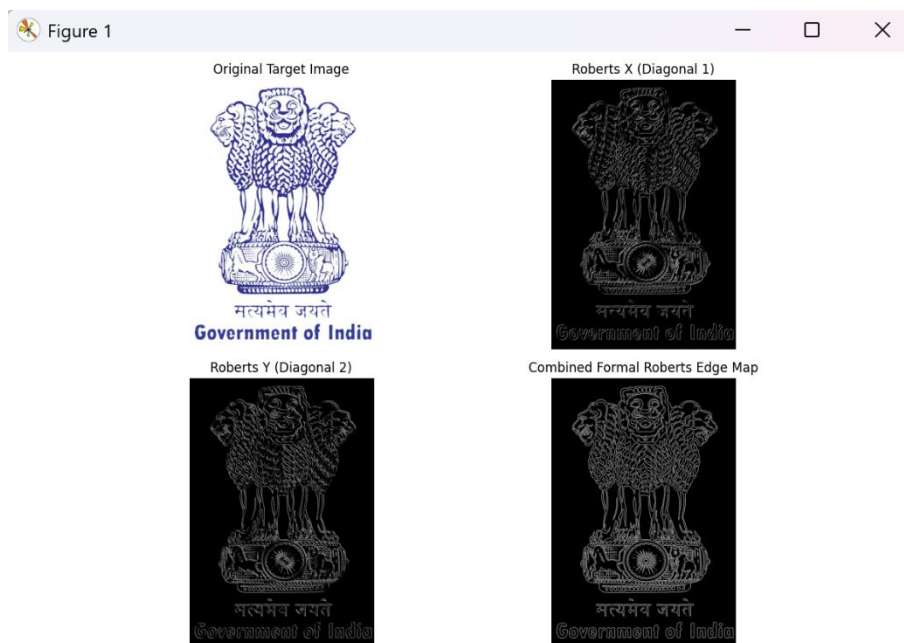
    plt.subplot(2, 2, 4)
    plt.imshow(edges, cmap='gray')
    plt.title('Combined Formal Roberts Edge Map')

```

```
plt.axis('off')

plt.tight_layout()
plt.show()

except ImportError:
    # Standard OpenCV fallback loop
    print("\nPlotting via Matplotlib failed. Launching OpenCV GUI... Close windows to exit.")
    cv2.imshow("Roberts X", grad_x)
    cv2.imshow("Roberts Y", grad_y)
    cv2.imshow("Combined Roberts Edge Map", edges)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```



Noise Models

Most images are degraded by noise during their acquisition and/or transmission. Many things may affect the hardware performance of image acquisition/transmission tools. For example, the lighting and temperature of the image sensors in the charge coupled device (CCD) camera are two major parameters which generate noise in an acquired image. Researchers assume that noise is an independent function of spatial coordinates and it is uncorrelated to the image. The spatial noise descriptor gives the statistical behavior of noise intensity. This intensity can be considered as a random variable and is given by the Probability Density Function (PDF). There are six main types of noise sources

that degrade images, which are explained in the following sections. The concept of generation of noise PDFs is beautifully described by Gonzales and Wood.

Gaussian Noise

Gaussian noise is a type of statistical noise with a probability density function equal to the normal distribution, often called a bell curve. It is defined by its mean (usually zero) and variance, acting as an additive, natural-source interference common in digital images and electronic communication systems.

Key Aspects of Gaussian Noise:

Definition: Sample values are distributed around a mean value, usually with a variance representing the noise power.

Applications: It is used to model random, natural phenomena such as thermal noise in amplifiers, camera sensor noise, and channel noise.

Additive White Gaussian Noise (AWGN): A common, simple model in signal processing, combining Gaussian distribution with a flat frequency spectrum.

Characteristics: It represents the sum of numerous independent random variables, conforming to the Central Limit Theorem.

Generation Methods: Techniques to generate this noise include the Box-Muller Transform and Marsaglia Polar Method.

Probability Distribution Function:

$$p(g) = \frac{1}{\sqrt{2\pi}\sigma} e^{-(n-\mu)^2/2\sigma^2}$$

where n is the noise intensity, μ is mean value of n , σ is standard deviation, and σ^2 is the variance of n .

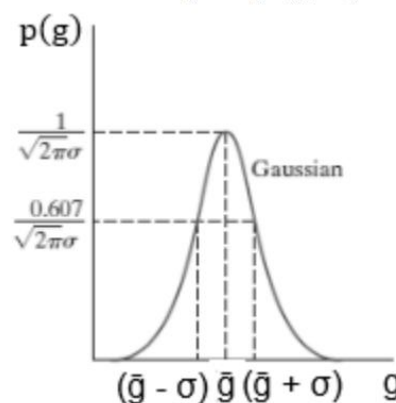


Fig: PDF of Gaussian Noise

Examples of Gaussian noise are noise due to electronic circuits and sensors.

Mean (μ): The average value of the noise, often zero.

Variance (σ^2): Determines the spread of the values and the intensity of the noise.

Standard Deviation (σ): Often used in software tools like PyTorch's Gaussian Noise to control noise magnitude

Uses:

Signal Processing: To test the robustness of signals.

Image Processing: Used for simulating sensor noise.

Machine Learning: Added as regularization in neural networks to prevent overfitting.

Python Code to illustrate Gaussian noise:

```
import cv2
import numpy as np
import tkinter as tk
from tkinter import filedialog
import matplotlib.pyplot as plt
import sys
import os

def open_file_dialog():
    """Opens a native OS file upload dialog box securely to get an image path"""
    print("Launching File Upload Dialog Box...")
    root = tk.Tk()
    root.withdraw()
    root.attributes('-topmost', True)

    file_path = filedialog.askopenfilename(
        title="Select an Input Image",
        filetypes=[
            ("Image Files", "*.png *.jpg *.jpeg *.bmp *.tiff"),
            ("All Files", "*.*")
        ]
    )
    root.destroy()
    return file_path

def add_gaussian_noise(image, mean=0, std_dev=25):
    """
    Generates and adds Gaussian noise to an image.
    :param image: Input original image.
    :param mean: Mean of the Gaussian distribution.
    :param std_dev: Standard deviation (spread) of the noise.
    """
    # Generate Gaussian noise matching the shape of the image
    gaussian_noise = np.random.normal(mean, std_dev, image.shape)

    # Add the generated noise to the original image
    noisy_image = image + gaussian_noise

    # Values might exceed [0, 255] bounds after adding noise, so we clip them
    noisy_image = np.clip(noisy_image, 0, 255)

    # Convert safely back to 8-bit unsigned integer (standard image format)
    return noisy_image.astype(np.uint8)
```

```

def calculate_histogram(image):
    """Calculates exactly the 2D grayscale histogram for an image"""
    # Flatten the 2D image matrix into a 1D array of active pixel values
    flattened_pixels = image.flatten()
    return flattened_pixels

if __name__ == "__main__":
    # 1. Input image from computer using Tkinter File Upload Dialog Box
    user_input = open_file_dialog()

    if not user_input or not os.path.exists(user_input):
        print("No valid file selected. Generating a generic gradient box to test
noise...")
        test_img = np.zeros((300, 300), dtype=np.uint8)
        # Create a smooth gradient background
        for i in range(300):
            test_img[:, i] = int((i / 300) * 255)
        # Add a block in the middle
        cv2.rectangle(test_img, (100, 100), (200, 200), (128), -1)

        cv2.imwrite("test_gradient.jpg", test_img)
        user_input = "test_gradient.jpg"

    # 2. Read the image uniformly into Grayscale
    # (Checking noise histograms is most accurate and standard on grayscale
layers)
    original_image = cv2.imread(user_input, cv2.IMREAD_GRAYSCALE)
    if original_image is None:
        print(f'Error: Could not load '{user_input}'. Must be an image file.')
        sys.exit(1)
    print(f'Processing image: {user_input}')

    # 3. Add Gaussian Noise manually
    noisy_image = add_gaussian_noise(original_image, mean=0, std_dev=40)

    # 4. Save both outputs dynamically to your workspace folder
    cv2.imwrite("gaussian_noise_output.jpg", noisy_image)
    print("\nSaved output image internally as 'gaussian_noise_output.jpg'")

    # 5. Extract Histograms
    hist_original = calculate_histogram(original_image)
    hist_noisy = calculate_histogram(noisy_image)

    # 6. Display the Images alongside their Histograms visually
    plt.figure(figsize=(14, 10))
    plt.suptitle("Gaussian Noise Illustration using Histograms", fontsize=16)

    # Top Left: Original Image
    plt.subplot(2, 2, 1)
    plt.imshow(original_image, cmap='gray')

```

```

plt.title('Original Image (Without Noise)')
plt.axis('off')

# Top Right: Noisy Image
plt.subplot(2, 2, 2)
plt.imshow(noisy_image, cmap='gray')
plt.title('Image WITH Gaussian Noise')
plt.axis('off')

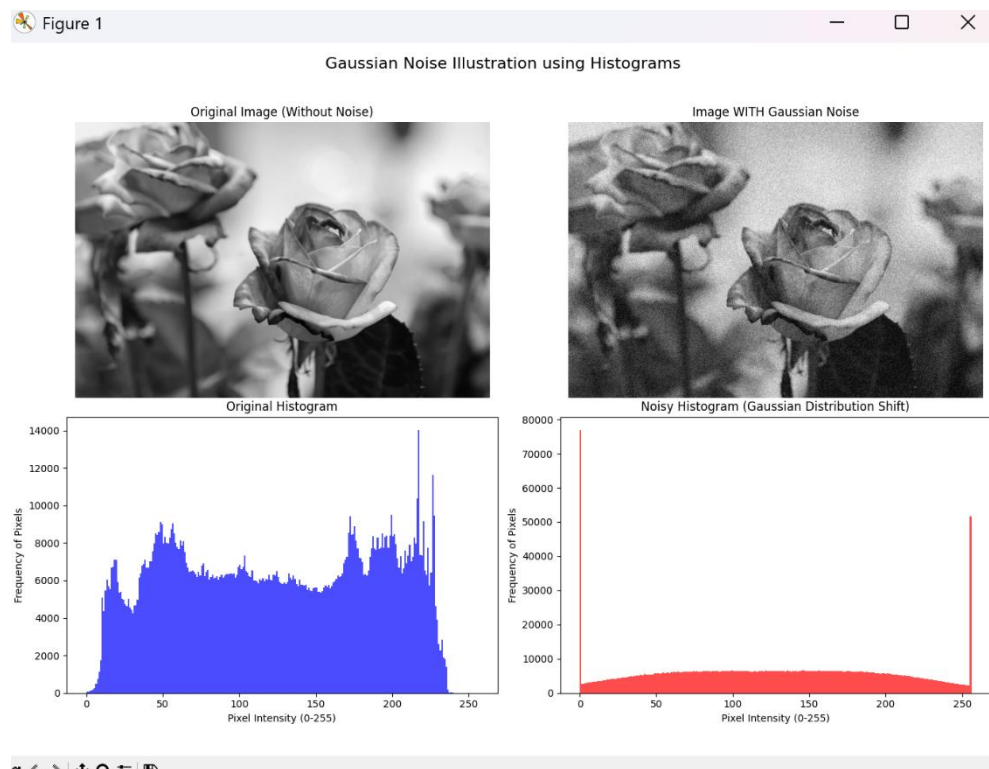
# Bottom Left: Original Histogram
plt.subplot(2, 2, 3)
plt.hist(hist_original, bins=256, range=[0, 256], color='blue', alpha=0.7)
plt.title('Original Histogram')
plt.xlabel('Pixel Intensity (0-255)')
plt.ylabel('Frequency of Pixels')

# Bottom Right: Noisy Histogram
plt.subplot(2, 2, 4)
plt.hist(hist_noisy, bins=256, range=[0, 256], color='red', alpha=0.7)
plt.title('Noisy Histogram (Gaussian Distribution Shift)')
plt.xlabel('Pixel Intensity (0-255)')
plt.ylabel('Frequency of Pixels')

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
print("Launching Matplotlib graph view... Close the popup window to exit.")
plt.show()

```

OUTPUT:



Rayleigh Noise

Rayleigh noise in image processing is a skewed, non-Gaussian additive noise model typically found in radar range and velocity images. Characterized by a Rayleigh distribution, it affects image quality with a specific probability density function (PDF), starting at a minimum value 'a' and exhibiting a, right-skewed peak, often modeled for noise reduction in range imagery.

Key Characteristics of Rayleigh Noise

Source: Primarily appears in radar, sonar, and range imaging due to scattering and signal interference.

Distribution: Unlike symmetric Gaussian noise, Rayleigh noise is skewed and non-symmetric

Probability Density Function(PDF):

$$p(g) = \begin{cases} \left(\frac{2}{b}\right)(n-a)e^{-(n-a)\frac{2}{b}}, & n \geq a \\ 0, & \text{otherwise} \end{cases}$$

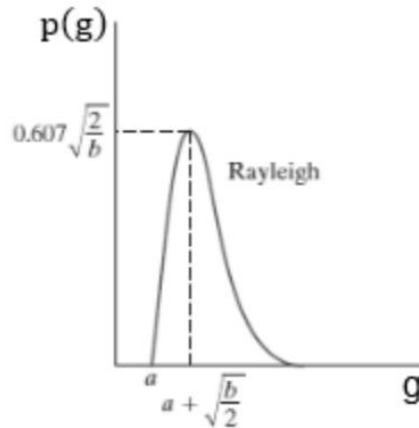


Fig: PDF for Rayleigh Noise

Mean and Variance:

$$\text{Here, } \mu = a + \sqrt{\pi b/4} \text{ and } \sigma^2 = \frac{b(4-\mu)}{4}.$$

Key Applications of Rayleigh Noise in Image Processing:

1. Medical Imaging (Ultrasound & MRI): Rayleigh distribution is commonly used to model the speckle noise that degrades Ultrasound and MRI images. Analysing this noise helps to develop filters that enhance image quality, aiding in better diagnostic accuracy.
2. Radar and Range Imaging: In radar signal processing, Rayleigh noise models the background clutter and noise in the signal, as it accurately represents the amplitude fluctuations of received signals.
3. Image Denoising/Restoration: By modelling the noise characteristics of specific sensors, filters can be tailored to remove Rayleigh-distributed

noise. Techniques like the Rayleigh-Monte Carlo method are used for image noise detection and elimination, providing higher precision in restoring images.

4. Algorithm Validation: Researchers use Rayleigh noise to create synthetic noisy images, allowing them to test the robustness and effectiveness of different denoising algorithms.
5. Range Data Modelling: In 3D imaging and LiDAR, Rayleigh noise can describe the uncertainties in distance calculations for images, which is essential for improving 3D mapping and object recognition.

Python Code to illustrate Rayleigh Noise is given below:

```
import cv2
import numpy as np
import tkinter as tk
from tkinter import filedialog
import matplotlib.pyplot as plt
import sys
import os

def open_file_dialog():
    """Opens a native OS file upload dialog box securely to get an image path"""
    print("Launching File Upload Dialog Box...")
    root = tk.Tk()
    root.withdraw()
    root.attributes('-topmost', True)

    file_path = filedialog.askopenfilename(
        title="Select an Input Image",
        filetypes=[
            ("Image Files", "*.png *.jpg *.jpeg *.bmp *.tiff"),
            ("All Files", "*.*")
        ]
    )
    root.destroy()
    return file_path

def add_rayleigh_noise(image, scale=30):
    """
    Generates and adds Rayleigh noise to an image.
    :param image: Input original image.
    :param scale: The scale parameter of the Rayleigh distribution.
    """
    # Generate Rayleigh noise matching the shape of the image
    rayleigh_noise = np.random.rayleigh(scale, image.shape)

    # Add the generated noise to the original image
    noisy_image = image + rayleigh_noise

    # Values might exceed [0, 255] bounds after adding noise, so we clip them
    noisy_image = np.clip(noisy_image, 0, 255)
```

```

# Convert safely back to 8-bit unsigned integer (standard image format)
return noisy_image.astype(np.uint8)

def calculate_histogram(image):
    """Calculates exactly the 2D grayscale histogram for an image"""
    # Flatten the 2D image matrix into a 1D array of active pixel values
    flattened_pixels = image.flatten()
    return flattened_pixels

if __name__ == "__main__":
    # 1. Input image from computer using Tkinter File Upload Dialog Box
    user_input = open_file_dialog()
    if not user_input or not os.path.exists(user_input):
        print("No valid file selected. Generating a generic gradient box to test
noise...")
        test_img = np.zeros((300, 300), dtype=np.uint8)
        # Create a smooth gradient background
        for i in range(300):
            test_img[:, i] = int((i / 300) * 255)
        # Add a block in the middle
        cv2.rectangle(test_img, (100, 100), (200, 200), (128), -1)
        cv2.imwrite("test_gradient_rayleigh.jpg", test_img)
        user_input = "test_gradient_rayleigh.jpg"
    # 2. Read the image uniformly into Grayscale
    original_image = cv2.imread(user_input, cv2.IMREAD_GRAYSCALE)
    if original_image is None:
        print(f"Error: Could not load '{user_input}'. Must be an image file.")
        sys.exit(1)
    print(f"Processing image: {user_input}")

    # 3. Add Rayleigh Noise manually
    # The scale parameter determines the offset and spread of the noise skew
    noisy_image = add_rayleigh_noise(original_image, scale=40)

    # 4. Save both outputs dynamically to your workspace folder
    cv2.imwrite("rayleigh_noise_output.jpg", noisy_image)
    print("\nSaved output image internally as 'rayleigh_noise_output.jpg'")

    # 5. Extract Histograms
    hist_original = calculate_histogram(original_image)
    hist_noisy = calculate_histogram(noisy_image)

    # 6. Display the Images alongside their Histograms visually
    plt.figure(figsize=(14, 10))
    plt.suptitle("Rayleigh Noise Illustration using Histograms", fontsize=16)

    # Top Left: Original Image
    plt.subplot(2, 2, 1)
    plt.imshow(original_image, cmap='gray')

```

```

plt.title('Original Image (Without Noise)')
plt.axis('off')

# Top Right: Noisy Image
plt.subplot(2, 2, 2)
plt.imshow(noisy_image, cmap='gray')
plt.title('Image WITH Rayleigh Noise')
plt.axis('off')

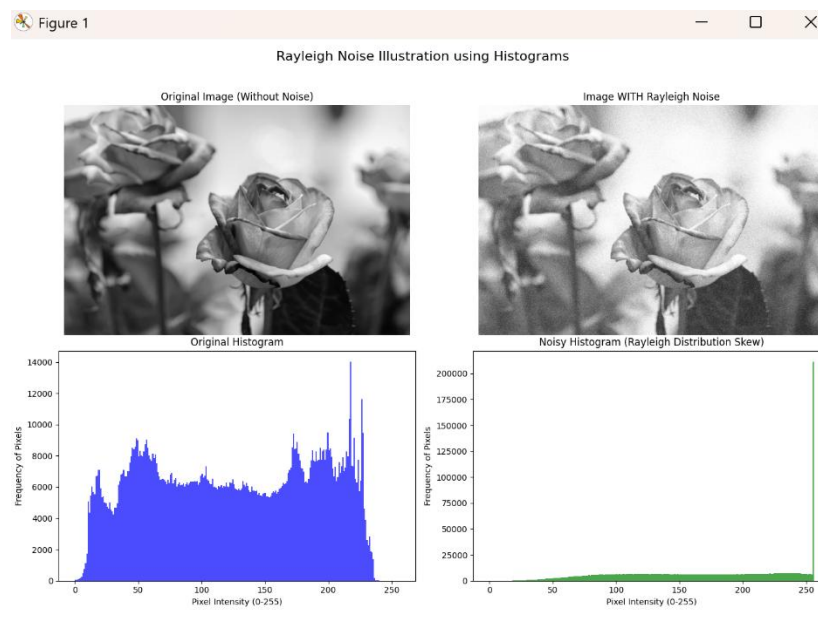
# Bottom Left: Original Histogram
plt.subplot(2, 2, 3)
plt.hist(hist_original, bins=256, range=[0, 256], color='blue', alpha=0.7)
plt.title('Original Histogram')
plt.xlabel('Pixel Intensity (0-255)')
plt.ylabel('Frequency of Pixels')

# Bottom Right: Noisy Histogram
plt.subplot(2, 2, 4)
plt.hist(hist_noisy, bins=256, range=[0, 256], color='green', alpha=0.7)
plt.title('Noisy Histogram (Rayleigh Distribution Skew)')
plt.xlabel('Pixel Intensity (0-255)')
plt.ylabel('Frequency of Pixels')

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
print("Launching Matplotlib graph view... Close the popup window to exit.")
plt.show()

```

OUTPUT



Erlang/Gamma Noise

Erlang (or Gamma) noise is a type of statistical noise used in digital image processing, particularly common in laser-based and X-ray imaging, where it

represents sensor deficiencies. It follows a specific probability density function (PDF) that is skewed to the right, often modeling intensity fluctuations. It depends on two parameters: a shape parameter b (positive integer) and a rate parameter a (positive real number). It is a "one-sided" distribution that begins at zero, unlike Gaussian noise. It is a generalization of exponential noise; if $b=1$ the expression reduces to exponential noise. The noise often appears as granular, light/dark spots due to the intensity values, and its effects can be analyzed via histogram evaluation in image processing.

The Erlang noise PDF is given as follows:

$$p(g) = \frac{a^b g^{b-1}}{(b-1)!}, g \geq 0$$

$$= 0, \text{ otherwise}$$

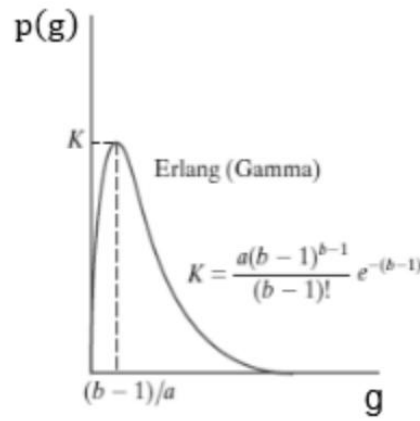


Fig: PDF for Erlang Noise

Mean and Variance:

Here, $\mu = \frac{b}{a}, \sigma^2 = \frac{b}{a^2}$ and $a > 0$, and b is a positive integer.

Applications of Erlang/Gamma Noise:

1. Laser and Active Imaging Simulation: It is highly effective for simulating noise patterns found in images acquired via laser systems and other high-intensity light-based imaging technologies.
2. Laser Imaging Noise Modelling: Erlang noise is used to model the noise in laser-based images, often where the sensor captures data with a positive skew, which Gaussian models cannot adequately represent.
3. Image Restoration and Filtering: The model is utilized in the design of specialized spatial filters (e.g., non-linear filters) aimed at reducing speckle-like noise while preserving, or even enhancing, image sharpness in fields like medical, satellite, and radar imaging.

4. Image Transmission/Storage Analysis: Due to its flexibility (via parameters 'a' and 'b'), this noise model is applied to analyse and improve image quality in communication channels where transmission produces non-symmetric, non-Gaussian noise.
5. Modelling Speckle in Coherent Imaging: The Gamma distribution is often used to model speckle noise in synthetic aperture radar (SAR) or sonar imaging, representing the multiplicative noise that occurs.

Python Code to illustrate Erlang Noise:

```
import cv2
import numpy as np
import tkinter as tk
from tkinter import filedialog
import matplotlib.pyplot as plt
import sys
import os

def open_file_dialog():
    """Opens a native OS file upload dialog box securely to get an image path"""
    print("Launching File Upload Dialog Box...")
    root = tk.Tk()
    root.withdraw()
    root.attributes('-topmost', True)
    file_path = filedialog.askopenfilename(
        title="Select an Input Image",
        filetypes=[
            ("Image Files", "*.png *.jpg *.jpeg *.bmp *.tiff"),
            ("All Files", "*.*")
        ]
    )
    root.destroy()
    return file_path

def add_erlang_noise(image, shape_k=2.0, scale_theta=15.0):
    """
    Generates and adds Erlang (Gamma) noise to an image.
    Erlang noise is a special case of Gamma noise where the shape parameter (k)
    is an integer.
    :param image: Input original image.
    :param shape_k: Shape parameter (k) - Must be an integer > 0 for pure Erlang
    noise.
    :param scale_theta: Scale parameter (theta or mean of exponential).
    """
    # Generate Erlang/Gamma noise matching the shape of the image
    # Note: numpy.random.gamma takes scale as the second argument
    erlang_noise = np.random.gamma(shape=shape_k, scale=scale_theta,
    size=image.shape)

    # Add the generated noise to the original image
    noisy_image = image + erlang_noise
    # Values might exceed [0, 255] bounds after adding noise, so we clip them
```

```

noisy_image = np.clip(noisy_image, 0, 255)
# Convert safely back to 8-bit unsigned integer (standard image format)
return noisy_image.astype(np.uint8)
def calculate_histogram(image):
    """Calculates exactly the 2D grayscale histogram for an image"""
    # Flatten the 2D image matrix into a 1D array of active pixel values
    flattened_pixels = image.flatten()
    return flattened_pixels
if __name__ == "__main__":
    # 1. Input image from computer using Tkinter File Upload Dialog Box
    user_input = open_file_dialog()
    if not user_input or not os.path.exists(user_input):
        print("No valid file selected. Generating a generic gradient box to test
noise...")
        test_img = np.zeros((300, 300), dtype=np.uint8)
        # Create a smooth gradient background
        for i in range(300):
            test_img[:, i] = int((i / 300) * 255)
        # Add a block in the middle
        cv2.rectangle(test_img, (100, 100), (200, 200), (128), -1)
        cv2.imwrite("test_gradient_erlang.jpg", test_img)
        user_input = "test_gradient_erlang.jpg"
    # 2. Read the image uniformly into Grayscale
    original_image = cv2.imread(user_input, cv2.IMREAD_GRAYSCALE)
    if original_image is None:
        print(f"Error: Could not load '{user_input}'. Must be an image file.")
        sys.exit(1)
    print(f"Processing image: {user_input}")
    # 3. Add Erlang Noise manually
    # We use a shape_k of 2 (integer) and a scale of 20 to make the noise visible
    noisy_image = add_erlang_noise(original_image, shape_k=2, scale_theta=20)
    # 4. Save both outputs dynamically to your workspace folder
    cv2.imwrite("erlang_noise_output.jpg", noisy_image)
    print("\nSaved output image internally as 'erlang_noise_output.jpg'")
    # 5. Extract Histograms
    hist_original = calculate_histogram(original_image)
    hist_noisy = calculate_histogram(noisy_image)
    # 6. Display the Images alongside their Histograms visually
    plt.figure(figsize=(14, 10))
    plt.suptitle("Erlang (Gamma) Noise Illustration using Histograms",
fontsize=16)
    # Top Left: Original Image
    plt.subplot(2, 2, 1)
    plt.imshow(original_image, cmap='gray')
    plt.title('Original Image (Without Noise)')
    plt.axis('off')

    # Top Right: Noisy Image
    plt.subplot(2, 2, 2)
    plt.imshow(noisy_image, cmap='gray')

```

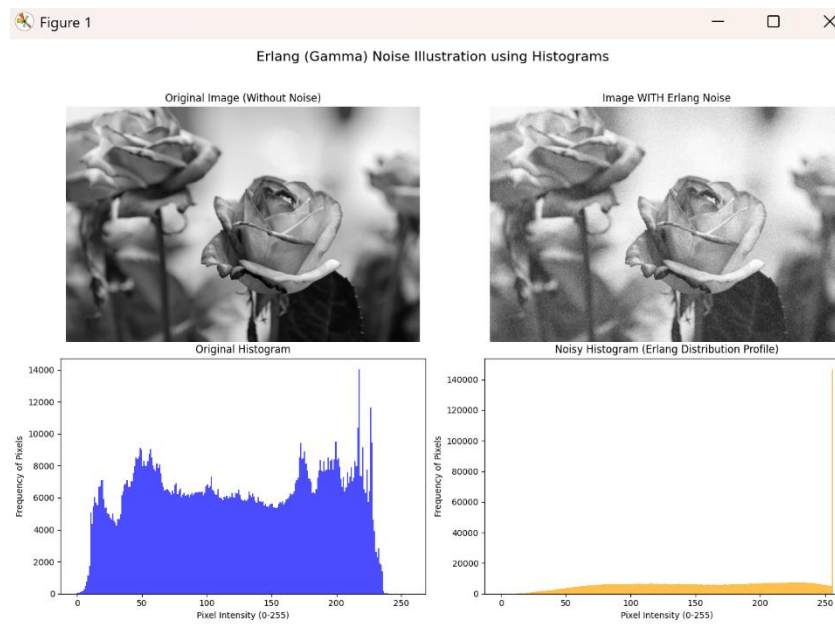
```
plt.title('Image WITH Erlang Noise')
plt.axis('off')

# Bottom Left: Original Histogram
plt.subplot(2, 2, 3)
plt.hist(hist_original, bins=256, range=[0, 256], color='blue', alpha=0.7)
plt.title('Original Histogram')
plt.xlabel('Pixel Intensity (0-255)')
plt.ylabel('Frequency of Pixels')

# Bottom Right: Noisy Histogram
plt.subplot(2, 2, 4)
plt.hist(hist_noisy, bins=256, range=[0, 256], color='orange', alpha=0.7)
plt.title('Noisy Histogram (Erlang Distribution Profile)')
plt.xlabel('Pixel Intensity (0-255)')
plt.ylabel('Frequency of Pixels')

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
print("Launching Matplotlib graph view... Close the popup window to exit.")
plt.show()
```

OUTPUT



Exponential Noise

Exponential noise in image processing is an additive, signal-dependent noise model, often simulating laser or photon emission imaging issues. It is a special

case of Erlang noise defined by an exponential decay probability density function. Exponential noise is a special case of Erlang/gamma noise with $b = 1$.

The noise PDF is given as follows:

$$p(g) = \begin{cases} a \cdot e^{-an}, & n \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

Mean and Variance:

Here, $\mu = \frac{1}{a}, \sigma^2 = \frac{1}{a^2}, a > 0$.

Applications of Exponential Noise:

1. Modelling Laser Imaging Noise: Exponential noise is frequently observed in images produced by laser systems.
2. Simulation of Acquisition Noise: It is used to simulate noise in image acquisition systems affected by poor illumination, high ISO settings, and heat.
3. Denoising Filter Testing: Exponential noise models are crucial for testing the performance of spatial filters. Order-statistic filters (such as median filters) are effective in reducing this type of noise.
4. Image Enhancement Evaluation: Researchers use synthetic exponential noise to challenge image enhancement algorithms, evaluating their ability to remove noise while preserving features, as it better simulates non-Gaussian noise compared to traditional Gaussian models.
5. Histogram Analysis: Because this noise follows a specific exponential decay pattern, it is used to teach histogram analysis and noise reduction through statistical modelling of image degradation.

Python Code to Illustrate Exponential Noise:

```
import cv2
import numpy as np
import tkinter as tk
from tkinter import filedialog
import matplotlib.pyplot as plt
import sys
import os

def open_file_dialog():
    """Opens a native OS file upload dialog box securely to get an image path"""
    print("Launching File Upload Dialog Box...")
    root = tk.Tk()
```

```

root.withdraw()
root.attributes('-topmost', True)

file_path = filedialog.askopenfilename(
    title="Select an Input Image",
    filetypes=[
        ("Image Files", "*.png *.jpg *.jpeg *.bmp *.tiff"),
        ("All Files", "*.*")
    ]
)
root.destroy()
return file_path

def add_exponential_noise(image, scale=30.0):
    """
    Generates and adds Exponential noise to an image.
    :param image: Input original image.
    :param scale: Scale parameter (inverse of the rate parameter lambda).
    """
    # Generate Exponential noise matching the shape of the image
    exponential_noise = np.random.exponential(scale=scale, size=image.shape)

    # Add the generated noise to the original image
    noisy_image = image + exponential_noise

    # Values might exceed [0, 255] bounds after adding noise, so we clip them
    noisy_image = np.clip(noisy_image, 0, 255)

    # Convert safely back to 8-bit unsigned integer (standard image format)
    return noisy_image.astype(np.uint8)

def calculate_histogram(image):
    """Calculates exactly the 2D grayscale histogram for an image"""
    # Flatten the 2D image matrix into a 1D array of active pixel values
    flattened_pixels = image.flatten()
    return flattened_pixels

if __name__ == "__main__":
    # 1. Input image from computer using Tkinter File Upload Dialog Box
    user_input = open_file_dialog()

    if not user_input or not os.path.exists(user_input):
        print("No valid file selected. Generating a generic gradient box to test noise...")

```

```

test_img = np.zeros((300, 300), dtype=np.uint8)
# Create a smooth gradient background
for i in range(300):
    test_img[:, i] = int((i / 300) * 255)
# Add a block in the middle
cv2.rectangle(test_img, (100, 100), (200, 200), (128), -1)

cv2.imwrite("test_gradient_exponential.jpg", test_img)
user_input = "test_gradient_exponential.jpg"

# 2. Read the image uniformly into Grayscale
original_image = cv2.imread(user_input, cv2.IMREAD_GRAYSCALE)
if original_image is None:
    print(f"Error: Could not load '{user_input}'. Must be an image file.")
    sys.exit(1)

print(f"Processing image: {user_input}")

# 3. Add Exponential Noise manually
# The scale defines the spread/intensity of the noise
noisy_image = add_exponential_noise(original_image, scale=45.0)

# 4. Save both outputs dynamically to your workspace folder
cv2.imwrite("exponential_noise_output.jpg", noisy_image)
print("\nSaved output image internally as 'exponential_noise_output.jpg'")

# 5. Extract Histograms
hist_original = calculate_histogram(original_image)
hist_noisy = calculate_histogram(noisy_image)

# 6. Display the Images alongside their Histograms visually
plt.figure(figsize=(14, 10))
plt.suptitle("Exponential Noise Illustration using Histograms", fontsize=16)

# Top Left: Original Image
plt.subplot(2, 2, 1)
plt.imshow(original_image, cmap='gray')
plt.title('Original Image (Without Noise)')
plt.axis('off')

# Top Right: Noisy Image
plt.subplot(2, 2, 2)
plt.imshow(noisy_image, cmap='gray')
plt.title('Image WITH Exponential Noise')

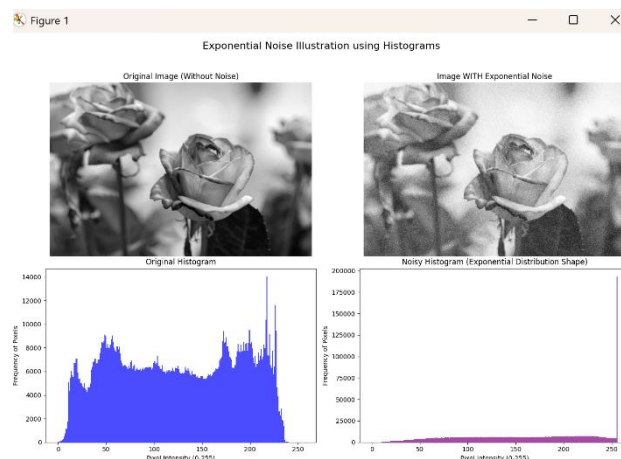
```

```
plt.axis('off')

# Bottom Left: Original Histogram
plt.subplot(2, 2, 3)
plt.hist(hist_original, bins=256, range=[0, 256], color='blue', alpha=0.7)
plt.title('Original Histogram')
plt.xlabel('Pixel Intensity (0-255)')
plt.ylabel('Frequency of Pixels')

# Bottom Right: Noisy Histogram
plt.subplot(2, 2, 4)
plt.hist(hist_noisy, bins=256, range=[0, 256], color='purple', alpha=0.7)
plt.title('Noisy Histogram (Exponential Distribution Shape)')
plt.xlabel('Pixel Intensity (0-255)')
plt.ylabel('Frequency of Pixels')

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
print("Launching Matplotlib graph view... Close the popup window to exit.")
plt.show()
```



Uniform or quantization Noise

Uniform noise is a type of signal interference where noise values are evenly distributed over a specific range. It is characterized by a rectangular probability density function and is primarily used in simulations to model quantization errors or in situations with poor lighting. Unlike Gaussian noise, uniform noise does not have a "bell curve" shape, but rather a flat, rectangular distribution. It is used to simulate random noise in an image where intensity values are uniformly distributed. While Gaussian noise has higher probabilities near the mean, uniform noise has equal probability across its entire range. It is inherently present in amplitude quantization process. It is caused by quantizing the pixels of a sensed image to several discrete levels. It obeys uniform distribution.

The uniform noise PDF is given as follows:

$$p(g) = \frac{1}{b-a}, a \leq n \leq b$$
$$= 0, \text{ otherwise}$$

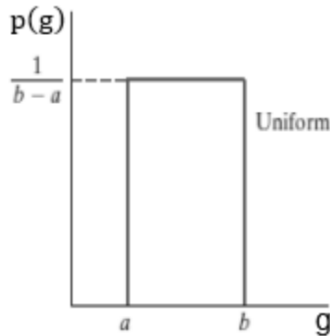


Fig: PDF for Uniform Noise

Mean and Variance:

$$\text{Here } \mu = \frac{a+b}{2}, \sigma^2 = \frac{(b-a)^2}{12}.$$

Applications of Uniform Noise:

1. Quantization Error Modelling: Uniform noise is used to model the error that occurs during uniform quantization in digital systems, where the difference between the actual signal and the quantized value is often modelled as having a uniform distribution.
2. Image Processing and Restoration: It is applied in digital image processing to simulate noise added to images, serving as a basis for testing image enhancement, restoration, and denoising algorithms.
3. Audio Dithering: Uniformly distributed noise can be used in digital audio to perform dithering, which reduces distortion by adding noise to a signal to remove quantization patterns.
4. Random Number Generation: Uniformly distributed noise is used to generate random numbers in computer simulations and stochastic modelling.
5. Neural Network Training (UNIQ): Uniform noise injection is used in machine learning, specifically in neural network quantization (e.g., [UNIQ] methods), where it is injected into weights during training to improve resilience to low-bit quantization.
6. Privacy Preservation: Uniform noise is employed in data mining and data analysis to add uncertainty and preserve privacy in datasets.

Python Code to illustrate Uniform Noise:

```
import cv2
import numpy as np
```

```

import tkinter as tk
from tkinter import filedialog
import matplotlib.pyplot as plt
import sys
import os

def open_file_dialog():
    """Opens a native OS file upload dialog box securely to get an image path"""
    print("Launching File Upload Dialog Box...")
    root = tk.Tk()
    root.withdraw()
    root.attributes('-topmost', True)

    file_path = filedialog.askopenfilename(
        title="Select an Input Image",
        filetypes=[
            ("Image Files", "*.png *.jpg *.jpeg *.bmp *.tiff"),
            ("All Files", "*.*")
        ]
    )
    root.destroy()
    return file_path

def add_uniform_noise(image, low=-50, high=50):
    """
    Generates and adds Uniform noise to an image.
    :param image: Input original image.
    :param low: Lower boundary of the uniform distribution span.
    :param high: Upper boundary of the uniform distribution span.
    """
    # Generate Uniform noise matching the shape of the image
    # We use a distribution spanning negative and positive to subtly adjust pixel
intensities equally
    uniform_noise = np.random.uniform(low=low, high=high, size=image.shape)

    # Add the generated noise to the original grayscale image
    noisy_image = image + uniform_noise

    # Values might exceed [0, 255] bounds after calculating noise, so we clip them
securely
    noisy_image = np.clip(noisy_image, 0, 255)

    # Convert safely back to 8-bit unsigned integer (standard format)
    return noisy_image.astype(np.uint8)

def calculate_histogram(image):
    """Calculates exactly the 2D grayscale histogram for an image"""
    # Flatten the 2D image matrix into a 1D dimensional string of pixel values
    flattened_pixels = image.flatten()
    return flattened_pixels

```

```

if __name__ == "__main__":
    # 1. Input image from computer using Tkinter File Upload Dialog Box
    user_input = open_file_dialog()

    if not user_input or not os.path.exists(user_input):
        print("No valid file selected. Generating a generic gradient box to test
noise...")
        test_img = np.zeros((300, 300), dtype=np.uint8)
        # Create a smooth gradient background
        for i in range(300):
            test_img[:, i] = int((i / 300) * 255)
        # Add a solid visual block in the middle
        cv2.rectangle(test_img, (100, 100), (200, 200), (128, -1)

        cv2.imwrite("test_gradient_uniform.jpg", test_img)
        user_input = "test_gradient_uniform.jpg"

    # 2. Read the image uniformly into Grayscale
    original_image = cv2.imread(user_input, cv2.IMREAD_GRAYSCALE)
    if original_image is None:
        print(f"Error: Could not load '{user_input}'. Must be an image file.")
        sys.exit(1)

    print(f"Processing image: {user_input}")

    # 3. Add Uniform Noise manually
    # Low & High sets an absolute uniform threshold where all interference holds
equal weight
    noisy_image = add_uniform_noise(original_image, low=-60, high=60)

    # 4. Save both outputs dynamically to your workspace folder
    cv2.imwrite("uniform_noise_output.jpg", noisy_image)
    print("\nSaved output image internally as 'uniform_noise_output.jpg'")

    # 5. Extract Histograms
    hist_original = calculate_histogram(original_image)
    hist_noisy = calculate_histogram(noisy_image)

    # 6. Display the Images alongside their Histograms visually
    plt.figure(figsize=(14, 10))
    plt.suptitle("Uniform Noise Illustration using Histograms", fontsize=16)

    # Top Left: Original Image
    plt.subplot(2, 2, 1)
    plt.imshow(original_image, cmap='gray')
    plt.title('Original Image (Without Noise)')
    plt.axis('off')

    # Top Right: Noisy Image

```

```

plt.subplot(2, 2, 2)
plt.imshow(noisy_image, cmap='gray')
plt.title('Image WITH Uniform Noise')
plt.axis('off')

# Bottom Left: Original Histogram
plt.subplot(2, 2, 3)
plt.hist(hist_original, bins=256, range=[0, 256], color='blue', alpha=0.7)
plt.title('Original Histogram')
plt.xlabel('Pixel Intensity (0-255)')
plt.ylabel('Frequency of Pixels')

# Bottom Right: Noisy Histogram
plt.subplot(2, 2, 4)
plt.hist(hist_noisy, bins=256, range=[0, 256], color='teal', alpha=0.7)
plt.title('Noisy Histogram (Equal Spread Profile)')
plt.xlabel('Pixel Intensity (0-255)')
plt.ylabel('Frequency of Pixels')

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
print("Launching Matplotlib graph view... Close the popup window to exit.")
plt.show()

```

OUTPUT:



Salt and Pepper or Impulse Noise

Salt and pepper noise is a specific, common type of impulse noise (or spike noise) that appears as randomly distributed black (pepper) and white (salt) pixels in digital images. It is caused by sharp, sudden disturbances like dead camera sensor pixels, analog-to-digital converter errors, or transmission failures.

The image looks as if it has been sprinkled with white and black dots. It generally results from hardware defects or bit errors in transmission. The affected pixels

only take the minimum or maximum intensity values (e.g., 0 or 255 for 8-bit images). Salt-and-pepper noise can be caused by sharp and sudden disturbances in the image signal. These may be from transmission errors, corrupted pixel elements in the camera sensors, or faulty memory locations in the storage media.

The PDF of salt and pepper noise is given as follows:

$$\begin{aligned} p(g) &= P_a && \text{for } g = a \\ &= P_b && \text{for } g = b \\ &= 0 && \text{Otherwise} \end{aligned}$$

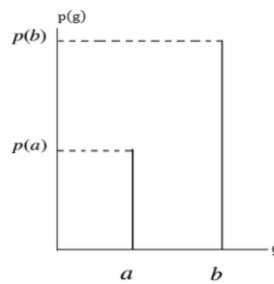


Fig: PDF for Salt and Pepper Noise

If $b > a$, then the intensity of b will appear as a white dot on the image; it appears as a black dot if P is zero. The noise is called a dot. Frequently, P_a and P_b are saturated values, resulting in positive impulses being a white dot and negative impulses being a black dot. This noise shows up when quick transitions take place in the image. If either P_a or P_b is zero, the impulse noise is called as unipolar, otherwise called as bipolar noise.

Applications of Salt and Pepper Noise:

1. Image Processing Validation: Used as a standard benchmark to test the effectiveness of filtering algorithms (e.g., Adaptive Median Filters) designed to restore images without blurring edges.
2. Modelling Data Transmission Errors: Simulates errors occurring during data transmission, such as packet loss, where pixels are lost and replaced with extreme maximum or minimum values.
3. Sensor Fault Analysis: Models defects in digital cameras where specific CCD pixels fail, producing consistent black or white spots.
4. Memory Fault Modelling: Simulates faulty memory locations within hardware that lead to corrupted, sparse data, such as a white (255) or black (0) dot.
5. Pattern Recognition & AI Training: Serves as a data augmentation technique for training convolutional neural networks (CNNs) and support vector machines (SVMs) to improve their robustness against noisy inputs.
6. Image Compression Testing: Applied during image compression to measure the stability and degradation of compression algorithms.

Python Code for Salt and Pepper Noise:

```
import cv2
import numpy as np
import tkinter as tk
from tkinter import filedialog
import matplotlib.pyplot as plt
import sys
import os
import random

def open_file_dialog():
    """Opens a native OS file upload dialog box securely to get an image path"""
    print("Launching File Upload Dialog Box...")
    root = tk.Tk()
    root.withdraw()
    root.attributes('-topmost', True)

    file_path = filedialog.askopenfilename(
        title="Select an Input Image",
        filetypes=[
            ("Image Files", "*.png *.jpg *.jpeg *.bmp *.tiff"),
            ("All Files", "*.*")
        ]
    )
    root.destroy()
    return file_path

def add_salt_and_pepper_noise(image, salt_prob=0.02, pepper_prob=0.02):
    """
    Generates and adds Salt and Pepper noise to an image.
    :param image: Input original image.
    :param salt_prob: Probability of a pixel becoming 'salt' (white/255).
    :param pepper_prob: Probability of a pixel becoming 'pepper' (black/0).
    """
    # Create a copy of the image to apply noise safely without modifying the
    original array
    noisy_image = np.copy(image)
    # Generate a random probability matrix the exact shape of the image
    probs = np.random.rand(noisy_image.shape[0], noisy_image.shape[1])

    # Apply Salt
    # Where the probability coordinate is less than salt_prob, turn pixel to
    maximum 255
    noisy_image[probs < salt_prob] = 255
    # Apply Pepper
    # Where probability coordinate lands in the specific upper margin, turn pixel
    to minimum 0
    noisy_image[probs > 1 - pepper_prob] = 0

    return noisy_image

def calculate_histogram(image):
```

```

"""Calculates exactly the 2D grayscale histogram for an image"""
# Flatten the 2D image matrix into a 1D dimensional string of pixel values
flattened_pixels = image.flatten()
return flattened_pixels

if __name__ == "__main__":
    # 1. Input image from computer using Tkinter File Upload Dialog Box
    user_input = open_file_dialog()
    if not user_input or not os.path.exists(user_input):
        print("No valid file selected. Generating a generic grayscale box to test
noise...")
        # A solid mid-gray image makes salt & pepper extremely easy to see in
histograms!
        test_img = np.full((300, 300), 128, dtype=np.uint8)
        cv2.imwrite("test_gray_salt_pepper.jpg", test_img)
        user_input = "test_gray_salt_pepper.jpg"
    # 2. Read the image uniformly into Grayscale
    original_image = cv2.imread(user_input, cv2.IMREAD_GRAYSCALE)
    if original_image is None:
        print(f'Error: Could not load '{user_input}'. Must be an image file.')
        sys.exit(1)
    print(f'Processing image: {user_input}')
    # 3. Add Salt and Pepper Noise manually
    # Combining the probabilities so roughly 10% of the image drops out
completely
    noisy_image = add_salt_and_pepper_noise(original_image, salt_prob=0.05,
pepper_prob=0.05)
    # 4. Save both outputs dynamically to your workspace folder
    cv2.imwrite("salt_pepper_noise_output.jpg", noisy_image)
    print("\nSaved output image internally as 'salt_pepper_noise_output.jpg'")
    # 5. Extract Histograms
    hist_original = calculate_histogram(original_image)
    hist_noisy = calculate_histogram(noisy_image)

    # 6. Display the Images alongside their Histograms visually
    plt.figure(figsize=(14, 10))
    plt.suptitle("Salt & Pepper Impulse Noise Illustration using Histograms",
fontsize=16)

    # Top Left: Original Image
    plt.subplot(2, 2, 1)
    plt.imshow(original_image, cmap='gray')
    plt.title('Original Image (Without Noise)')
    plt.axis('off')

    # Top Right: Noisy Image
    plt.subplot(2, 2, 2)
    plt.imshow(noisy_image, cmap='gray')
    plt.title('Image WITH Salt & Pepper Noise')
    plt.axis('off')

```

```

# Bottom Left: Original Histogram
plt.subplot(2, 2, 3)
plt.hist(hist_original, bins=256, range=[0, 256], color='blue', alpha=0.7)
plt.title('Original Histogram')
plt.xlabel('Pixel Intensity (0-255)')
plt.ylabel('Frequency of Pixels')

# Bottom Right: Noisy Histogram
plt.subplot(2, 2, 4)
plt.hist(hist_noisy, bins=256, range=[0, 256], color='black', alpha=0.7)
plt.title('Noisy Histogram (Spikes at 0 and 255)')
plt.xlabel('Pixel Intensity (0-255)')
plt.ylabel('Frequency of Pixels')

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
print("Launching Matplotlib graph view... Close the popup window to exit.")
plt.show()

```

OUTPUT:

